Persistent Community Search in Temporal Networks

Rong-Hua Li[†], Jiao Su[*], Lu Qin[‡], Jeffrey Xu Yu[*], Xiaokui Xiao[§], and Qiangqiang Dai[†]

[†] *Shenzhen University, Shenzhen, China;* [*]*The Chinese University of Hong Kong, Hong Kong*
[‡]*Centre for QCIS, FEIT, University of Technology, Sydney, Australia;* [§]*Nanyang Technological University, Singapore*
{rhli,mao}@szu.edu.cn; {jiaosu, yu}@se.cuhk.edu.hk;
Lu.Qin@uts.edu.au; xkxiao@ntu.edu.sg; qiang56734@163.com

ABSTRACT

Community search is a fundamental graph mining task. In applications such as analysis of communication networks, collaboration networks, and social networks, edges are typically associated with timestamps. Unfortunately, most previous studies focus mainly on identifying communities in a network without temporal information. In this paper, we study the problem of finding persistent communities in a temporal network, in which every edge is associated with a timestamp. Our goal is to identify the communities that are persistent over time. To this end, we propose a novel persistent community model called (θ, τ)-persistent k-core to capture the persistence of a community. We prove that the problem of identifying the maximum (θ, τ)-persistent k-core is NP-hard. To solve this problem, we first propose a near-linear temporal graph reduction algorithm to prune the original temporal graph substantially, without loss of accuracy. Then, in the reduced temporal graph, we present a novel branch and bound algorithm with several carefully-designed pruning rules to find the maximum (θ, τ)-persistent k-cores efficiently. We conduct extensive experiments in several real-world temporal networks. The results demonstrate the efficiency, scalability, and effectiveness of the proposed solutions.

1. INTRODUCTION

Many real-world networks such as social networks, biological networks, and communication networks contain community structures. Finding communities in a network is a fundamental task in network analysis which has attracted much attention in recent years due to a large number of real-life applications [5, 27, 15, 21].

In applications such as analysis of communication networks, collaboration networks of scientific papers, and social networks, the edges are often associated with temporal information. For example, in the mobile-phone calling network, each phone call contains a sender and a receiver, as well as the time when the phone call was made. In the face-to-face contact network [24, 9], each edge (u, v, t) represents a contact between two individuals u and v at time t. Another example is the collaboration network, where each edge consists of two authors and the time when they coauthored a paper. Unfortunately, most previous community search algorithms ignore the temporal information of the graph, thus may fail to identify important temporal patterns such as the persistent and evolving community structures.

In this paper, we investigate the problem of finding persistent community structures in the temporal network, where each edge is associated with a timestamp. We aim to discover the communities in a temporal network that are persistent over time. To this end, an intuitive approach is to identify all the *persistent edges* in the temporal network (e.g., the edge with persistence no less than a given threshold), and then applies traditional community search algorithms in the temporal sub-network induced by those *persistent edges* to find the communities. This method, however, cannot completely reveal the persistent communities. This is because various *persistent edges* may be persistent over diverse time intervals, and thus the whole community is not necessary persistent over time. Indeed, we empirically show that such an intuitive solution fails to identify persistent communities (see Section 5.2 for details). Another potential approach is the temporal clique enumeration [25, 26]. As defined in [25, 26], a temporal clique involves a set of nodes and a time interval I, and it persistently maintains the clique structure in any θ-length subinterval of I. However, in this model, the *clique constraint* could be too strong, which may miss many useful persistent communities violating such a *clique constraint*. Moreover, the temporal clique model only considers one time interval. The real-world persistent communities, however, may be persistent over different time intervals. Therefore, the temporal clique model cannot fully capture the persistence of the communities.

To overcome the limitations of the above methods, we propose a new persistent community model, called (θ, τ)-persistent k-core, based on the concept of k-core [22]. For a time interval I, our model persistently preserves the k-core structure in any θ-length subinterval of I. We refer to such an interval I as a persistent interval for convenience. A persistent interval is maximal if none of its super-intervals is a persistent interval. A community may have many different maximal persistent intervals. We thus aggregate the duration of all such maximal persistent intervals to measure the *core persistence* of a community. We define a community as a (θ, τ)-persistent k-core if it is a maximal induced temporal subgraph such that its *core persistence* is no smaller than a given parameter τ (see Section 2 for details). In many temporal network applications, (θ, τ)-persistent k-core can help to discover interesting groups that are persistent over time. Such persistent groups can be useful for finding a stable team of experts in collaboration networks, and revealing different contact patterns in face-to-face contact networks [25]. Below, we detail the applications.

Application I. Consider an application of finding a team of experts in a scientific collaboration network (e.g., DBLP). We may wish

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Copyright 20XX ACM X-XXXXX-XX-X/XX/XX ...$15.00.

to find a *stable* and cohesive team to complete a task in which the members continuously coauthored papers with each other over a prolonged period. As mentioned above, an intuitive approach for this application is to identify all the long-term coauthors for each author, and then applies traditional community model (e.g., k-core) to detect communities over those long-term links. However, this intuitive method may not be sufficient to discover stable and cohesive teams. This is because the cohesive structure of a team could not be persistently preserved, albeit each link in the team is persistent over time. Finding the (θ, τ)-persistent k-cores, however, can be useful for this application, as our model requires that the resulting teams persistently maintain a k-core structure for a long time.

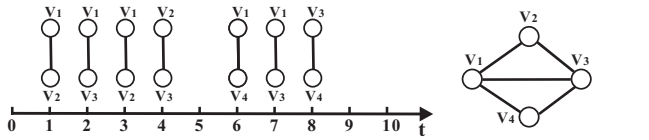
Application II. Consider a temporal network of face-to-face contacts between patients and health-care workers (including nurses, doctors, and administrative staffs) in a hospital [24]. Finding (θ, τ)-persistent k-cores in this temporal network can help to reveal the nature of interactions between patients and health-care workers, and show interesting semantics of contact. For example, let $\theta = 60$ seconds, $k = 3$, and $\tau = 300$ seconds. Suppose that we find a $(60, 300)$-persistent 3-core community with total duration 300 seconds involving a patient, two nurses and a doctor. This community is likely to be an event that the doctor treats the patient with the help of two nurses, as a typical treatment process may last around 5 minutes. Note that traditional community models (e.g., k-core) cannot reveal such a persistent contact pattern. Another example is that a $(300, 3600)$-persistent 10-core community with duration 3600 seconds, including an administrative staff and 10 nurses, probably corresponds to a one-hour meeting among these individuals. Further, assume that we find a $(3600, 28800)$-persistent 3-core community including four nurses during more than 8 hours. Then, this community is likely to be a group of nurses that work in the same room, because they persistently contact each other in a prolonged period. Note that revealing such important contact patterns in a hospital can be useful for preventing the spread of hospital-acquired infections [24]. For instance, consider again the $(3600, 28800)$-persistent 3-core community involving four nurses. If one of them get an infectious disease, then the other three nurses are at a high risk of getting the same infectious disease.

Contributions. In this paper, we formalize and provide solutions to find the persistent communities in temporal networks. In particular, we make the following principal contributions.

New model and hardness result. We propose a new persistent community model called (θ, τ)-persistent k-core to fully characterize the persistence of a community. We prove that computing the maximum (θ, τ)-persistent k-core (or enumerating all (θ, τ)-persistent k-cores) is NP-hard.

Novel algorithms. To find the maximum (θ, τ)-persistent k-core, we first develop a novel temporal graph reduction (TGR) algorithm with near-linear time complexity to prune the input temporal graph. The TGR algorithm is based on an elegant and newly-developed meta-interval decomposition technique. We show that computing the meta-interval decomposition in a temporal graph can be done in linear time and uses linear space. Then, in the reduced temporal graph, we propose a novel branch and bound algorithm with several powerful pruning rules to find the maximum (θ, τ)-persistent k-core efficiently. Moreover, our algorithm can also be used to enumerate all (θ, τ)-persistent k-cores.

Experimental evaluation. We conduct extensive experiments to evaluate the proposed model and algorithms using several real-world temporal networks. The results show that in a temporal graph with more than one million nodes and ten millions edges, our algorithm consumes less than 100 seconds to identify the maximum (θ, τ)-persistent k-core in most parameter settings. We also perform comprehensive case studies to evaluate the effectiveness of the proposed model. The results demonstrate that our model can identify many meaningful and interesting persistent communities that cannot be found by the other models.



(a) A temporal graph (b) The projected graph

Figure 1: Running example

The rest of this paper is organized as follows. We propose the persistent community model and formulate our problem in Section 2. We present the temporal graph reduction technique and the branch and bound algorithm in Sections 3 and 4 respectively. The experimental results are reported in Section 5. Finally, we survey the related work and conclude this paper in Section 6 and Section 7 respectively.

2. PRELIMINARIES

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected temporal graph, where $\mathcal{V}$ and $\mathcal{E}$ denote the set of vertices and the set of temporal edges respectively. Let $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$ be the number of nodes and temporal edges respectively. Each temporal edge $e \in \mathcal{E}$ is a triplet (u, v, t), where u, v are vertices in V, and t is the interaction time between u and v. Let st and ed be the time of the first and the last occurred temporal edges respectively. The interaction time t of any temporal edge (u, v, t) in $\mathcal{G}$ must fall into the interval $[st, ed]$. The length of the interval $[st, ed]$ is equal to $|ed - st|$. In this paper, we assume that t is an integer, because the timestamp is an integer in practice. Note that the temporal edges (u, v, t_1) and (u, v, t_2) with $t_2 \neq t_1$ are considered as two different temporal edges. Clearly, the temporal graph can be represented as edge streams [26, 12]. Fig. 1(a) illustrates an example of a temporal graph with 4 nodes and 7 temporal edges.

For a subset of vertices $\mathcal{V}_S$, the induced temporal subgraph $\mathcal{G}_S = (\mathcal{V}_S, \mathcal{E}_S)$ is a subgraph such that $\mathcal{E}_S = \{(u, v, t) | u, v \in \mathcal{V}_S, (u, v, t) \in \mathcal{E}\}$. The temporal neighborhood of a node u in $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is defined by $\mathcal{N}_u(\mathcal{G}) \triangleq \{(u, v, t) | (u, v, t) \in \mathcal{E}\}$. For a temporal graph $\mathcal{G}$, the projected graph denoted by G over the time interval $[t_s, t_e]$ is defined as $G = (V, E, [t_s, t_e])$, where $V = \mathcal{V}$ and $E = \{(u, v) | (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. Fig. 1(b) illustrates the projected graph of the temporal graph in Fig. 1(a) over the interval $[1, 8]$. The degree of a node u in G, denoted by $d_u(G)$, is the number of neighbors in G. A k-core in a graph G is an induced subgraph $C = (V_C, E_C)$, where each node in V_C has a degree no smaller than k [22]. A *connected* k-core is a k-core and it is also connected.

The persistent community model. Intuitively, a persistent community should continue to occur over a prolonged period in the temporal graph. To model a community, we choose the well-known k-core model [22], as it is widely used in community search applications [23, 18, 29]. Based on this, we propose a novel persistent community model, called (θ, τ)-persistent k-core, which can capture the persistence of the k-core structure.

For convenience, we assume that both the parameters θ and τ in our model are integers, because the timestamp is an integer. Note that the proposed model and algorithms can also be extended to handle the case when θ and τ are non-integers. Below, we first give a definition called maximal (θ, k)-persistent-core interval.

DEFINITION 1. (Maximal (θ, k)-persistent-core interval) *Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and parameters θ and k, an interval $[t_s, t_e]$ with $t_e - t_s \geq \theta$ is called a maximal (θ, k)-persistent-core interval for $\mathcal{G}$ if and only if the following two conditions hold. (1) For any $t \in [t_s, t_e - \theta]$, the projected graph of $\mathcal{G}$ over the interval $[t, t + \theta]$ is a* connected *k-core. (2) There is no super-interval of $[t_s, t_e]$ such that (1) holds.*

By Definition 1, we can see that the temporal graph $\mathcal{G}$ in the maximal (θ, k)-persistent-core interval persistently maintains the k-core structure in any θ-length subinterval.

EXAMPLE 1. *Consider the temporal graph $\mathcal{G}$ in Fig. 1(a). Assume that $\theta = 3$ and $k = 2$. We can see that there is no maximal $(3, 2)$-persistent-core interval for the entire graph $\mathcal{G}$. However, for the subgraph $\mathcal{C}$ induced by nodes $\{v_1, v_2, v_3\}$, there is a maximal $(3, 2)$-persistent-core interval $[1, 5]$, because $[1, 5]$ is the maximal interval such that in any 3-length subinterval of $[1, 5]$, the nodes $\{v_1, v_2, v_3\}$ form a connected 2-core. However, if $\theta = 1$ and $k = 2$, we can easily obtain that the subgraph $\mathcal{C}$ has three maximal $(1, 2)$-persistent-core intervals which are $[1, 2]$, $[2, 3]$, and $[3, 4]$ respectively.*

Note that there may be several maximal (θ, k)-persistent-core intervals for a temporal graph $\mathcal{G}$, and these intervals may overlap each other. However, the following lemma shows that the length of the overlapped subintervals of any two maximal (θ, k)-persistent-core intervals must be smaller than θ. All the proofs can be found in Appendix A.1.

LEMMA 1. *Let $[t_{s_1}, t_{e_1}]$ and $[t_{s_2}, t_{e_2}]$ be two overlapped maximal (θ, k)-persistent-core intervals of a temporal graph $\mathcal{G}$, and $t_{e_1} < t_{e_2}$. Then, we have $t_{e_1} - t_{s_2} \leq \theta$.*

Based on the maximal (θ, k)-persistent-core interval, we define the core persistence of a temporal graph $\mathcal{G}$ as follows.

DEFINITION 2. (Core persistence) *Let $\mathcal{T} = \{[t_{s_1}, t_{e_1}], \cdots, [t_{s_r}, t_{e_r}]\}$ be the set of all maximal (θ, k)-persistent-core intervals of $\mathcal{G}$. Then, the core persistence of $\mathcal{G}$ with parameters θ and k, denoted by $F(\theta, k, \mathcal{G})$, is defined as*

$$F(\theta, k, \mathcal{G}) \triangleq \begin{cases} \sum_{i=1}^{r} (t_{e_i} - t_{s_i}) - (r-1)\theta, & if \mathcal{T} \neq \emptyset \\ 0, & otherwise \end{cases} \quad (1)$$

By Definition 2, the core persistence of a temporal graph $\mathcal{G}$ is equal to the total length of all maximal (θ, k)-persistent-core intervals of $\mathcal{G}$ minus $(r-1)\theta$. The rationale behind this definition is twofold. First, since two maximal (θ, k)-persistent-core intervals may overlap and the length of the overlapped interval is bounded by θ (see Lemma 1), we discount θ for every two *adjacent* intervals. For r maximal (θ, k)-persistent-core intervals, the total length thus decreases by $(r-1)\theta$. Second, using Eq. (1) can reduce the effect of the θ-length interval. Consider the following two cases: (1) there are r θ-length maximal (θ, k)-persistent-core intervals for $\mathcal{G}$, and (2) there is only one maximal (θ, k)-persistent-core interval for $\mathcal{G}$ with length $r \times \theta$. Intuitively, $\mathcal{G}$ under the case (2) should be considered as more persistent than $\mathcal{G}$ under the case (1). This is because in case (2), $\mathcal{G}$ maintains the k-core structure in a r times longer interval than each interval in case (1). By using the total length of all intervals, we cannot distinguish these two different cases. By Definition 2, the core persistence of $\mathcal{G}$ in the case (1) is θ, whereas in the case (2) it equals $r \times \theta$. Based on these reasons, we use Definition 2 to define the core persistence. It is worth mentioning that the proposed algorithms also work when the core persistence is defined as the total length of all maximal (θ, k)-persistent-core intervals.

DEFINITION 3. ((θ, τ)-persistent k-core) *Given a temporal graph $\mathcal{G}$, parameters θ, τ, and k, a (θ, τ)-persistent k-core is an induced temporal subgraph $\mathcal{C} = (\mathcal{V}_C, \mathcal{E}_C)$ that meets the following properties.*

- Persistent core property*: $F(\theta, k, \mathcal{C}) \geq \tau$;*
- Maximal property*: there does not exist an induced temporal subgraph $\mathcal{C}'$ that contains $\mathcal{C}$ and also satisfies the persistent core property.*

Intuitively, the (θ, τ)-persistent k-core is capable of capturing the persistence of a community, because it persistently preserves a k-core structure in every θ time in the maximal (θ, k)-persistent-core intervals with total length no less than τ.

EXAMPLE 2. *Reconsider the temporal graph $\mathcal{G}$ in Fig. 1(a). Let $\theta = 3$, $\tau = 4$ and $k = 2$. We claim that the subgraph $\mathcal{C}$ induced by nodes $\{v_1, v_2, v_3\}$ is a $(3, 4)$-persistent 2-core. The reason is as follows. First, the subgraph $\mathcal{C}$ has a maximal $(3, 2)$-persistent-core interval $[1, 5]$ (see Example 1). Therefore, the core persistence of $\mathcal{C}$ is equal to 4 (i.e., $F(3, 2, \mathcal{C}) = 4$), which is no less than τ. On the other hand, $\mathcal{C}$ is the maximal subgraph that meets such a persistent core property. As a consequence, the subgraph $\mathcal{C}$ is a $(3, 4)$-persistent 2-core.*

Note that the (θ, τ)-persistent k-cores of a temporal graph may overlap each other. As a result, the number of (θ, τ)-persistent k-cores may be exponentially large, and thus it is very costly to enumerate all (θ, τ)-persistent k-cores. In practice, we are often interested in finding the maximum (θ, τ)-persistent k-core. Therefore, in this paper, we focus mainly on finding the largest (θ, τ)-persistent k-core. We emphasize that the proposed solutions can also be used to enumerate all (θ, τ)-persistent k-cores as discussed in Section 4.

The persistent community (PC) search problem. Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, parameters θ, τ and k, we aim to find the largest (θ, τ)-persistent k-core in $\mathcal{G}$.

Hardness result. Since the number of (θ, τ)-persistent k-cores in a temporal graph can be exponentially large, it is very hard to identify the largest one. We show that the PC search problem is NP-hard.

THEOREM 1. *The problem of finding the largest (θ, τ)-persistent k-core is NP-hard.*

Comparison with the temporal clique model. Our (θ, τ)-persistent k-core model is closely related to the temporal clique model [12]. As defined in [26], a temporal clique comprises a set of nodes V and a time interval I (both V and I are maximal), where every pair of nodes are connected at least once in each θ-length subinterval of I. In other words, the temporal clique persistently preserves the clique structure in any θ-length subinterval of I [26]. The limitations of the temporal clique model are threefold. First, the temporal clique model cannot fully capture the persistence of a community, as it only considers a persistent interval I. In many real-world applications, however, the community may be persistent over different time intervals. Second, in the temporal clique model, the constraint is too strict, as it requires every pair of node must contact once in every θ-length time interval. This may miss many useful persistent communities that do not satisfy such a *clique* constraint. Third, the state-of-the-art algorithm to enumerate the temporal cliques is very costly [12], which is intractable for large temporal graphs.

Unlike the temporal clique, the (θ, τ)-persistent k-core model can capture the persistence of the community over different time intervals. Our model relaxes the clique constraint, thus it can detect persistent communities that cannot be found by the temporal clique model as confirmed in our experiment (see Section 5). Moreover, we develop a tractable algorithm with a novel temporal graph reduction technique to find the persistent communities based on our model. In Appendix A.2, we also revise the temporal clique model to capture the persistence of the community over different time intervals based on a similar *interval aggregation* method as used in Eq. (1). The proposed temporal graph reduction technique can also be used to speed up the *persistent clique* search.

3. TEMPORAL GRAPH REDUCTION

As shown in Theorem 1, the problem of finding the largest (θ, τ)-persistent k-core is NP-hard. Therefore, straightforwardly computing the largest (θ, τ)-persistent k-core is intractable for large temporal graphs. To solve our problem efficiently, we propose a novel temporal graph reduction (TGR) technique, which can substantially reduce the size of the temporal graph without missing any (θ, τ)-persistent k-core. We show that our TGR technique can be implemented in near-linear time (with a factor θ), thus it is very efficient in practice. After reducing the temporal graph size, we will propose an efficient branch and bound algorithm to solve our problem in Section 4.

3.1 The key idea

By Definition 3, we can easily derive that every node in a (θ, τ)-persistent k-core must persistently preserve the property of "degree no less than k" in the maximal (θ, k)-persistent-core intervals with total length no smaller than τ. For convenience, we refer to such a property as a *persistent-degree* property. Clearly, we can safely prune the nodes in the temporal graph $\mathcal{G}$ that do not meet the *persistent-degree* property. Furthermore, pruning the unpromising nodes may result in their neighbor nodes that do not satisfy the *persistent-degree* property. Therefore, we can iteratively remove the unpromising nodes from $\mathcal{G}$ until all nodes in the reduced graph satisfy the persistent-degree property. Below, we introduce two new definitions, which are important to devise the temporal graph reduction (TGR) algorithm.

DEFINITION 4. (Maximal (θ, k)-persistent-degree interval) *Given a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a node $u \in \mathcal{V}$, and two integer parameters θ and k, an interval $[t_s^u, t_e^u]$ with $t_e^u - t_s^u \geq \theta$ is called a maximal (θ, k)-persistent-degree interval for u if and only if the following two conditions hold. (1) For any $t \in [t_s^u, t_e^u - \theta]$, the degree of u in the projected graph of $\mathcal{G}$ over the interval $[t, t+\theta]$ is no less than k. (2) There does not exist a super-interval of $[t_s^u, t_e^u]$ such that (1) holds.*

In the maximal (θ, k)-persistent-degree interval of u, the degree of u is no smaller than k in any θ-length subinterval by Definition 4. In this sense, u persistently maintains the property of "degree no less than k" in the maximal (θ, k)-persistent-degree interval.

EXAMPLE 3. *Consider the temporal graph in Fig. 1(a). Let $\theta = 3$ and $k = 2$. Then, there are two maximal (θ, k)-persistent-degree intervals for node v_1 which are $[-1, 5]$*[1] *and $[4, 9]$. This is because in $[-1, 5]$, v_1 has a degree no less than 2 in each subinterval with length 3. Moreover, there is no super-interval of $[-1, 5]$ such that v_1 persistently maintains the property of degree no less than 2. Note that the interval $[-1, 6]$ is not a maximal (θ, k)-persistent-degree interval, as there exist θ-length subintervals (e.g., $[2.5, 5.5]$) such that v_1's degree is smaller than 2. Similar results also hold for the interval $[4, 9]$. Clearly, we can see that these two maximal (θ, k)-persistent-degree intervals overlap. Likewise, for node v_2, we can obtain a maximal (θ, k)-persistent-degree interval $[1, 6]$. For node v_3, we obtain two maximal (θ, k)-persistent-degree intervals which are $[1, 5]$ and $[5, 10]$. For v_4, we have a maximal (θ, k)-persistent-degree interval $[5, 9]$.*

[1]Note that -1 as a *timestamp* does not affect the correctness of all the examples given in this paper.

Similar to Definition 1, there may exist several maximal (θ, k)-persistent-degree intervals for a node u, and these intervals may overlap each other (as shown in Example 3). The following lemma shows that for a node u, the length of the overlapped subinterval of two maximal (θ, k)-persistent-degree intervals is smaller than θ.

LEMMA 2. *Let $[t_{s_1}^u, t_{e_1}^u]$ and $[t_{s_2}^u, t_{e_2}^u]$ be two overlapped maximal (θ, k)-persistent-degree intervals of a node u, and $t_{e_1}^u < t_{e_2}^u$. Then, we have $t_{e_1}^u - t_{s_2}^u \leq \theta$.*

Similar to Definition 2, we define the degree persistence for a node u as follows.

DEFINITION 5. (Degree persistence) *Let $\mathcal{T} = \{[t_{s_1}^u, t_{e_1}^u], \cdots, [t_{s_r}^u, t_{e_r}^u]\}$ be the set of all maximal (θ, k)-persistent-degree intervals of u. Then, the degree persistence of u with parameters θ and k, denoted by $f(u, \theta, k, \mathcal{G})$, is defined as*

$$f(u, \theta, k, \mathcal{G}) \triangleq \begin{cases} \sum_{i=1}^{r} (t_{e_i}^u - t_{s_i}^u) - (r-1)\theta, & if \mathcal{T} \neq \emptyset \\ 0, & otherwise. \end{cases} \quad (2)$$

By Definition 5, the degree persistence of each node satisfies *monotonic* property. That is to say, the degree persistence of any node u in a temporal graph $\mathcal{G}$ is no less than the degree persistence of u in any temporal subgraph of $\mathcal{G}$.

LEMMA 3. *Let $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ be a temporal subgraph of $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$. Then, for any node $u \in \mathcal{V}_1$, we have $f(u, \theta, k, \mathcal{G}_1) \leq f(u, \theta, k, \mathcal{G}_2)$.*

EXAMPLE 4. *Reconsider the temporal graph $\mathcal{G}$ in Fig. 1(a). Let $\theta = 3$ and $k = 2$. Then, based on the results in Example 3, we can easily derive that $f(v_1, 3, 2, \mathcal{G}) = 8$, $f(v_2, 3, 2, \mathcal{G}) = 5$, $f(v_3, 3, 2, \mathcal{G}) = 6$, and $f(v_4, 3, 2, \mathcal{G}) = 4$. However, if we only consider the temporal subgraph $\mathcal{G}'$ induced by the nodes $\{v_1, v_2, v_3\}$, we can obtain that $f(v_1, 3, 2, \mathcal{G}') = 6$, $f(v_2, 3, 2, \mathcal{G}') = 5$, $f(v_3, 3, 2, \mathcal{G}') = 4$. Clearly, we have $f(v_1, 3, 2, \mathcal{G}') < f(v_1, 3, 2, \mathcal{G})$, $f(v_2, 3, 2, \mathcal{G}') = f(v_2, 3, 2, \mathcal{G})$, and $f(v_3, 3, 2, \mathcal{G}') < f(v_3, 3, 2, \mathcal{G})$. This example confirms that the degree persistence of each node indeed satisfies the monotonic property (Lemma 3).*

By Definitions 3 and 5, we can easily prove that the degree persistence of each node in a (θ, τ)-persistent k-core must be no less than τ.

LEMMA 4. *Suppose that there is a (θ, τ)-persistent k-core $\mathcal{C}$ in the temporal graph $\mathcal{G}$. Then, for every node $u \in \mathcal{C}$, we has $f(u, \theta, k, \mathcal{C}) \geq \tau$.*

Based on Lemmas 3 and 4, we can safely prune the node v in $\mathcal{G}$ if $f(u, \theta, k, \mathcal{G}) < \tau$, as v is definitely not contained in any (θ, τ)-persistent k-core. Moreover, after pruning all the unpromising nodes in $\mathcal{G}$, the degree persistence of the residual nodes may decrease (by Lemma 3). Thus, this pruning procedure may trigger a *domino effect*, i.e., the pruning of unpromising nodes may result in new unpromising nodes. As a consequence, we can iteratively remove the unpromising nodes until the degree persistence of every node in the residual graph is no less than τ. We refer to such an iterative pruning procedure as a TGR procedure. The following example illustrates the TGR procedure.

Algorithm 1 Meta-interval-Decomposition $(u, \mathcal{N}_u, \theta)$

1: Let $\mathcal{N}_u = \{(u, v_1, t_1), \cdots, (u, v_h, t_h)\}$ with $v_i \neq v_j$ for $i \neq j$;
2: **for** $i = 1$ to h **do**
3: Let $[t_i, t_i + \theta]$ be the lifespan of (u, v_i, t_i);
4: $\mathcal{T}_1(i) \leftarrow \{t_i, +1\}; \mathcal{T}_2(i) \leftarrow \{t_i + \theta, -1\}$;
5: $\mathcal{T}' \leftarrow$ Sort$(\{\mathcal{T}_1, \mathcal{T}_2\}); i \leftarrow 1; j \leftarrow 1$;
6: **while** $i < |\mathcal{T}'| - 1$ **do**
7: $\mathcal{T}(j).first \leftarrow \mathcal{T}'(i).first; d \leftarrow \mathcal{T}'(i).second$;
8: **while** $i < |\mathcal{T}'| - 1$ and $\mathcal{T}'(i).first = \mathcal{T}'(i+1).first$ **do**
9: $d \leftarrow d + \mathcal{T}'(i+1).second; i \leftarrow i + 1$;
10: $\mathcal{T}(j).second \leftarrow d; j \leftarrow j + 1$;
11: $d \leftarrow 0$;
12: **for** $i = 1$ to $|\mathcal{T}| - 1$ **do**
13: $d \leftarrow d + \mathcal{T}(i).second$; {// compute the prefix sum for $\mathcal{T}$}
14: $\mathcal{D}_u(i) \leftarrow d; \mathcal{MI}_u(i) \leftarrow (\mathcal{T}(i).first, \mathcal{T}(i+1).first)$;
15: **return** $\{\mathcal{D}_u, \mathcal{MI}_u\}$;

EXAMPLE 5. *Reconsider the temporal graph $\mathcal{G}$ in Fig. 1(a). Suppose that $\theta = 3$, $k = 2$ and $\tau = 5$. As shown in Example 4, v_4 is an unpromising node, as $f(v_4, 3, 2, \mathcal{G}) = 4 < \tau$. Thus, we can remove v_4. After pruning v_4, we obtain a temporal subgraph $\mathcal{G}'$ induced by $\{v_1, v_2, v_3\}$. The deletion of v_4 leads to $f(v_3, 3, 2, \mathcal{G}') = 4 < \tau$ (Example 4), and thus we can further delete v_3. Iteratively, we can get that all nodes will be deleted, indicating that $\mathcal{G}$ does not contain a $(3, 5)$-persistent 2-core.*

Challenges. There are two challenges to implement the TGR procedure. First, we need to compute the degree persistence for each node u in $\mathcal{G}$ efficiently. By Definition 5, we has to find all the maximal (θ, k)-persistent-degree intervals of u to compute the degree persistence of u. It is nontrivial to devise an efficient algorithm to find all such intervals, because the intervals may overlap each other. Second, the degree persistence of a node u may frequently update in the TGR procedure. It is challenging to devise an efficient algorithm to maintain the degree persistence for the remaining nodes after pruning an unpromising node.

Below, we propose an elegant meta-interval decomposition technique to tackle these challenges. Based on the meta-interval decomposition technique, we can implement the TGR procedure in near-linear time, using linear space.

3.2 Meta-interval decomposition

We assume in this paper that all the temporal edges in $\mathcal{G}$ are sorted in an increasing order by their timestamps. Note that if this assumption does not hold, we can sort the temporal edges in linear time, because each timestamp is an integer.

By Definitions 4 and 5, we can compute the degree persistence for a node u based on the temporal neighborhood of u. Let $\mathcal{N}_u(\mathcal{G}) = \{(u, v_1, t_1), \cdots, (u, v_h, t_h)\}$ with $t_i < t_{i+1}$ $(i = 1, \cdots, h-1)$ be the temporal neighborhood of u. Note that all the $v_i \in \mathcal{N}_u(G)$ are not necessarily distinct, i.e., there may exist repeated neighbors. For example, in the temporal graph shown in Fig. 1(a), $\mathcal{N}_{v_1} = \{(v_1, v_2, 1), (v_1, v_3, 2), (v_1, v_2, 3), (v_1, v_4, 6), (v_1, v_3, 7)\}$ in which v_2 and v_3 are repeated neighbors of v_1. The degree of u w.r.t. $\mathcal{N}_u(\mathcal{G})$ denotes the number of distinct neighbors in $\mathcal{N}_u(\mathcal{G})$. To easily describe the meta-interval decomposition technique, we first assume that all the $v_i \in \mathcal{N}_u(G)$ are distinct, and then we will show how to extend our technique to handle the case when there exist repeated neighbors in $\mathcal{N}_u(G)$.

For each node u, the key idea of our technique is to decompose the entire time interval of $\mathcal{N}_u(G)$ into a set of meta-intervals, and then compute and maintain the degree persistence of u only using the meta-intervals. We introduce the definition of the meta-interval as follows.

DEFINITION 6. (Meta-Interval) *Given a node u and its temporal neighborhood $\mathcal{N}_u(\mathcal{G})$, an open interval (t_s, t_e) is a meta-interval of u if and only if the following conditions hold: (1) $t_e - t_s > \theta$, and (2) in every θ-length subinterval of (t_s, t_e), u has the same degree.*

Definition 6 indicates that a node u in the meta-interval (t_s, t_e) has a *uniform* degree. Below, we give a useful definition called θ-persistent degree for a node u in the interval $[t_l, t_r]$ (or (t_l, t_r)).

DEFINITION 7. (θ-persistent degree) *Given a node u and a temporal graph $\mathcal{G}$. The θ-persistent degree of a node u in the interval $I = [t_l, t_r]$ (or (t_l, t_r)) is the maximal integer d such that u has a degree no less than d in every θ-length subinterval of I.*

Clearly, in the meta-interval of u, the so-called *uniform* degree of u is equal to the θ-persistent degree by Definitions 6 and 7. However, it should be noted that the θ-persistent degree of a node u is also well-defined for any general interval (not only for meta-intervals). The following example illustrates the definitions of meta-interval and θ-persistent degree.

EXAMPLE 6. *Consider the node v_1 in the temporal graph in Fig. 1(a). Suppose that $\theta = 3$. Then, we can check that $(0, 5)$ is a meta-interval for v_1, because in any 3-length subinterval of $(0, 5)$, v_1 has the same degree 2. Similarly, the open interval $(2, 6)$ is also a meta-interval for v_1. This is because in the open interval $(2, 6)$, there are only two temporal edges ($(v_1, v_2, 3)$ and $(v_2, v_3, 4)$), and v_1 has the same degree 1 in any 3-length subinterval of $(2, 6)$. Clearly, by Definition 7, the 3-persistent degree of v_1 in the meta-interval $(0, 5)$ is 2, because 2 is the maximal integer such that v_1 has a degree no less than 2 in every 3-length subinterval of $(0, 5)$. Also, we can see that in the closed interval $[5, 9]$, the 3-persistent degree of v_1 is 2.*

To compute the meta-intervals of a node u and the corresponding θ-persistent degree in each meta-interval, we define the lifespan of a temporal edge (u, v, t) as $[t, t + \theta]$, because the edge (u, v, t) contributes 1 to the degree of u in this interval.

Algorithm 1 outlines our meta-interval decomposition technique, which can compute all the meta-intervals as well as the corresponding θ-persistent degrees for a node u. For each temporal edge $(u, v_i, t_i) \in \mathcal{N}_u(G)$, the algorithm generates two pairs $\{t_i, +1\}$ and $\{t_i + \theta, -1\}$ (lines 2-4), denoting that the edge (u, v_i) contributes 1 to the degree of u at time t_i and decreases the degree by 1 at time $t_i + \theta$. This is because the lifespan of each temporal edge (u, v_i, t_i) is $[t_i, t_i + \theta]$ by our definition. Then, in line 5, Algorithm 1 sorts all these pairs in an increasing order by their timestamps, i.e., the first item of each pair. Let $\mathcal{T}'$ be the sorted list of those pairs. For each $\mathcal{T}'(i)$, the first term $\mathcal{T}'(i).first$ is a timestamp, and the second term $\mathcal{T}'(i).second$ is an integer either 1 or -1. Note that the first terms of $\mathcal{T}'$ (i.e., all $\mathcal{T}'(i).first$) are not necessarily distinct. Algorithm 1 filters those repeated timestamps in $\mathcal{T}'$, and takes the sum of the second terms of $\mathcal{T}'$ at each repeated timestamp (lines 6-10). After this processing, the algorithm creates an array $\mathcal{T}$ to record the results, where all the first terms in $\mathcal{T}$ are distinct. Subsequently, the algorithm computes the prefix sum based on the second terms of $\mathcal{T}$ (line 13), and builds two arrays $\mathcal{D}_u$ and $\mathcal{MI}_u$ to record the prefix sums and the intervals $(\mathcal{T}(i).first, \mathcal{T}(i+1).first)$ for all $i = 1, \cdots, |\mathcal{T}| - 1$, respectively (line 14). By our construction, we can easily derive that the prefix sum computed from $\mathcal{T}(1)$ to $\mathcal{T}(i)$ is equal to the number of *surviving* edges at timestamp $\mathcal{T}(i).first$. Finally, the algorithm returns the arrays $\mathcal{D}_u$ and $\mathcal{MI}_u$. The following theorem

shows that for each interval $\mathcal{MI}_u(i) = (t_s, t_e)$, the θ-shifted interval $(t_s - \theta, t_e)$ is a meta-interval, and the corresponding θ-persistent degree of u in $(t_s - \theta, t_e)$ is $\mathcal{D}_u(i)$.

THEOREM 2. *For each $\mathcal{MI}_u(i) = (t_s, t_e)$ and $\mathcal{D}_u(i)$ computed by Algorithm 1, the interval $(t_s - \theta, t_e)$ is a meta-interval. Moreover, in any θ-length subinterval of $(t_s - \theta, t_e)$, u has the same degree $\mathcal{D}_u(i)$.*

Since each interval in $\mathcal{MI}_u$ corresponds to a meta-interval, we call $\mathcal{MI}_u$ the meta-interval array. We also refer to $\mathcal{D}_u$ as the θ-persistent degree array, because $\mathcal{D}_u(i)$ is equal to the θ-persistent degree of u in the meta-interval $\mathcal{MI}_u(i)$. The following example illustrates the meta-interval decomposition technique.

EXAMPLE 7. *Reconsider the temporal graph in Fig. 1(a). Let us consider the node v_4 with parameter $\theta = 3$. By Algorithm 1, we have $\mathcal{T}' = \mathcal{T} = \{(6, +1), (8, +1), (9, -1), (11, -1)\}$. Then, we can easily derive that the θ-persistent degree array (i.e., the prefix sum array) $\mathcal{D}_{v_4} = \{1, 2, 1\}$ and the meta-interval array $\mathcal{MI}_{v_4} = \{(6, 8), (8, 9), (9, 11)\}$. We can see that for any $\mathcal{MI}_{v_4}(i) = (t_s, t_e)$, the θ-persistent degree of v_4 in the meta-interval $(t_s - \theta, t_e)$ is $\mathcal{D}_{v_4}(i)$. For instance, in the meta-interval $(5, 9)$ (corresponding to $\mathcal{MI}_{v_4}(2)$), v_4 has a degree 2 in every subinterval with length 3.*

Handling repeated neighbors. Here we extend the meta-interval decomposition technique to the case when the node has repeated neighbors. First, we find that in $\mathcal{N}_u(G)$, if the time gap between any two repeated neighbors is no less than θ, we can deem them as different neighbors and use Algorithm 1 to compute the meta-intervals. The reason is as follows. Let (u, v, t_1) and (u, v, t_2) be two temporal edges, and v is the repeated neighbor of u in $\mathcal{N}_u(G)$. If the gap $t_2 - t_1 \geq \theta$, the next repeated neighbor (v, t_2) appears after (v, t_1) vanishing, thus they do not repeatedly contribute 1 to the degree in any θ-length interval.

Second, for two temporal edges (u, v, t_1) and (u, v, t_2) with $0 < t_2 - t_1 < \theta$, the lifespan of (u, v, t_1) and (u, v, t_2) is $[t_1, t_1 + \theta]$ and $[t_2, t_2 + \theta]$ respectively. Clearly, by our previous algorithm (Algorithm 1), the edges (u, v, t_1) and (u, v, t_2) will contribute 2 to the degree of u in the interval $[t_2, t_1 + \theta]$, which is incorrect. To remedy this issue, we revise the lifespan of (u, v, t_1) by $[t_1, t_2]$, and keep the lifespan of (u, v, t_2) unchanged. Note that this processing can be done in linear time. After this processing, we can use Algorithm 1 to compute the meta-intervals of u. Specifically, we only need to change the line 3 of Algorithm 1 using the revised lifespan for the repeated neighbors. The following example illustrates how the algorithm works.

EXAMPLE 8. *Consider the node v_1 in Fig. 1(a). We assume that $\theta = 3$. Clearly, for node v_1, the time gap between the two repeated neighbors $(v_2, 1)$ and $(v_2, 3)$ is smaller than θ. Thus, we need to revise the lifespan of the temporal edge $(v_1, v_2, 1)$ by $[1, 3]$. Note that for the two repeated neighbors $(v_3, 2)$ and $(v_3, 7)$, since the time gap between them is larger than θ, it is no need to update the lifespan of the temporal edge $(v_1, v_3, 2)$. After that, we can obtain that $\mathcal{T}' = \{(1, +1), (2, +1), (3, -1), (3, +1), (5, -1), (6, -1), (6, +1), (7, +1), (9, -1), (10, -1)\}$, and $\mathcal{T} = \{(1, +1), (2, +1), (3, 0), (5, -1), (6, 0), (7, +1), (9, -1), (10, -1)\}$. Then, we have $\mathcal{D}_{v_1} = \{1, 2, 2, 1, 1, 2, 1\}$ and $\mathcal{MI}_{v_1} = \{(1, 2), (2, 3), (3, 5), (5, 6), (6, 7), (7, 9), (9, 10)\}$.*

Complexity analysis. We analyze the time and space complexity to compute the meta-interval decomposition for all nodes in $\mathcal{G}$ using Algorithm 1 as follows.

THEOREM 3. *Both the time and space complexity of Algorithm 1 to compute the meta-interval decomposition for all nodes $u \in \mathcal{G}$ are $O(m)$, where m is the number of temporal edges.*

Discussions. The problem of computing the meta-interval decomposition is related to the temporal aggregation problem [31, 3, 17]. The proposed technique, however, is fundamentally different from the previous temporal aggregation techniques for temporal data. Specifically, previous temporal aggregation techniques focus mainly on partitioning the temporal interval and then building tree-shaped index structure to support fast temporal aggregation queries [31, 3, 17]. Our meta-interval decomposition technique aims to decompose the temporal interval into a set of meta-intervals such that the node has the same degree in each θ-length subinterval. The striking features of our meta-interval decomposition technique are twofold. First, it can be computed in linear time, using linear space. Second, based on the meta-interval decomposition, we are able to compute the degree persistence for all nodes in linear time (see Section 3.3), and update the degree persistence in $O(\theta)$ time (see Section 3.4).

3.3 Computing degree persistence

Recall that to compute the degree persistence for a node u, we need to find all the maximal (θ, k)-persistent-degree intervals of u. Here we show that all the maximal (θ, k)-persistent-degree intervals of u can be constructed by merging adjacent meta-intervals, and the degree persistence of u can also be computed by the meta-interval decomposition technique. Below, we consider two different cases.

Case I: $f(u, \theta, k, \mathcal{G}) > \theta$. If the degree persistence $f(u, \theta, k, \mathcal{G}) > \theta$, there must be a maximal (θ, k)-persistent-degree interval of u with length larger than θ. In this case, we can ignore all θ-length maximal (θ, k)-persistent-degree intervals of u, because these intervals contribute nothing to the degree persistence by Definition 5. Therefore, we focus on finding all maximal (θ, k)-persistent-degree intervals of u with length larger than θ.

Let $\mathcal{MI}_u = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$ be the meta-interval array of a node u calculated by Algorithm 1 with parameter θ, and $\mathcal{D}_u = \{\mathcal{D}_u(1), \cdots, \mathcal{D}_u(h)\}$ be the θ-persistent degree array of u. For convenience, we define $\mathcal{D}_u(0) = \mathcal{D}_u(h + 1) = -\infty$. Then, we have the following result.

LEMMA 5. *For a parameter k and $i \leq h-1$, if both $\mathcal{D}_u(i) \geq k$ and $\mathcal{D}_u(i + 1) \geq k$ hold, u has a degree no less than k in any θ-length subinterval of $[t_{s_i} - \theta, t_{e_{i+1}}]$.*

Lemma 5 indicates that we can merge two adjacent meta-intervals in $\mathcal{MI}_u$ if the θ-persistent degree of u in these intervals is no less than k. In the merged interval, u persistently maintains the property of "degree no less k". Based on Lemma 5, we can obtain the maximal (θ, k)-persistent-degree intervals of u by merging the adjacent meta-intervals in $\mathcal{MI}_u$.

THEOREM 4. *If $\mathcal{D}_u(i) \geq k$ for all $i = l, \cdots, r$ ($l \leq r$), $\mathcal{D}_u(l-1) < k$ and $\mathcal{D}_u(r + 1) < k$ hold, then $[t_{s_l} - \theta, t_{e_r}]$ is a maximal (θ, k)-persistent-degree interval of u.*

The following example illustrates the correctness of Theorem 4.

EXAMPLE 9. *Consider the node v_1 in Fig. 1(a). Let $k = 2$ and $\theta = 3$. Then, we have $\mathcal{D}_{v_1} = \{1, 2, 2, 1, 1, 2, 1\}$ and $\mathcal{MI}_{v_1} = \{(1, 2), (2, 3), (3, 5), (5, 6), (6, 7), (7, 9), (9, 10)\}$ (see Example 8). In $\mathcal{MI}_{v_1}$, we have two adjacent meta-intervals $(2, 3)$ and $(3, 5)$ such that v_1's θ-persistent degree in the intervals $(-1, 3)$*

and $(0,5)$ *is no less than* k *(*$k = 2$*). Also, we have a meta-interval* $(7,9)$ *such that* v_1*'s* θ*-persistent degree in the* θ*-shifted interval* $(4,9)$ *is no less than* 2*. By Theorem 4, we can obtain two maximal* (θ,k)*-persistent-degree intervals* $[-1,5]$ *and* $[4,9]$*, which are consistent with the results shown in Example 3.*

Theorem 4 shows that all maximal (θ,k)-persistent-degree intervals of u with length larger than θ can be obtained by meta-interval decomposition. Based on this, we can compute the degree persistence for every node u if $f(u,\theta,k,\mathcal{G}) > \theta$ by Theorem 5.

THEOREM 5. *For a node* u*, if* $f(u,\theta,k,\mathcal{G}) > \theta$*, we have* $f(u,\theta,k,\mathcal{G}) = \sum_{\mathcal{D}_u(i)\geqslant k}(t_{e_i} - t_{s_i}) + \theta$*, where* $(t_{s_i}, t_{e_i}) \in \mathcal{MI}_u$*.*

Case II: $f(u,\theta,k,\mathcal{G}) \leq \theta$**.** By Definition 5, if $f(u,\theta,k,\mathcal{G}) \leq \theta$, there is no maximal (θ,k)-persistent-degree interval of u with length larger than θ. We consider the following two cases. First, if $\tau > \theta$, we can immediately prune node u, because u's degree persistence $f(u,\theta,k,\mathcal{G}) \leq \theta < \tau$.

Second, if $\tau \leq \theta$, we can slightly modify Algorithm 1 to compute all the maximal (θ,k)-persistent-degree intervals of u. Note that in this case, the length of each maximal (θ,k)-persistent-degree interval of u must be equal to θ (because $f(u,\theta,k,\mathcal{G}) \leq \theta$). To compute the maximal (θ,k)-persistent-degree intervals of u, we create two additional arrays $\tilde{\mathcal{MI}}_u$ and $\tilde{\mathcal{D}}_u$. For each $\mathcal{T}(i)$ in the line 14 of Algorithm 1, we generate an additional closed *point-interval* $\tilde{\mathcal{MI}}_u(i) = [\mathcal{T}(i).first, \mathcal{T}(i).first]$ and calculate the degree $\tilde{\mathcal{D}}_u(i)$ which equals the number of surviving temporal edges at the timestamp $\mathcal{T}(i).first$. Note that this process can be done in linear time. Clearly, in the θ-shifted interval $[\mathcal{T}(i).first - \theta, \mathcal{T}(i).first]$, u has a degree $\tilde{\mathcal{D}}_u(i)$. Any maximal (θ,k)-persistent-degree interval must be a θ-shifted interval of a *point-interval* contained in $\tilde{\mathcal{MI}}_u$. Theorem 6 shows how to recognize the θ-length maximal (θ,k)-persistent-degree intervals from $\tilde{\mathcal{MI}}_u$ based on the arrays $\mathcal{D}_u$ and $\tilde{\mathcal{D}}_u$.

THEOREM 6. *For an interval* $\tilde{\mathcal{MI}}_u(i) = [t_i, t_i]$*, if* $\tilde{\mathcal{D}}_u(i) = k$*,* $\mathcal{D}_u(i-1) < k$ *and* $\mathcal{D}_u(i) < k$*, then* $[t_i - \theta, t_i]$ *is a* θ*-length maximal* (θ,k)*-persistent-degree interval of* u*.*

Based on the above results, we can find all maximal (θ,k)-persistent-degree intervals of u and compute $f(u,\theta,k,\mathcal{G})$ if $f(u,\theta,k,\mathcal{G}) \leq \theta$.

Complexity. Theorems 5 and 6 demonstrate that computing the degree persistence for all nodes in $\mathcal{G}$ can be done in linear time based on the meta-interval decomposition technique.

3.4 Updating degree persistence

For each node u, we can update the degree persistence of u when a temporal edge in $\mathcal{N}_u(\mathcal{G})$ is deleted or inserted based on the meta-interval decomposition. In the rest of this paper, we focus mainly on the case of $f(u,\theta,k,\mathcal{G}) > \theta$, and the proposed algorithms can easily extend to handle the $f(u,\theta,k,\mathcal{G}) \leq \theta$ case. Let $\mathcal{MI}_u = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$ and $\mathcal{D}_u$ be the arrays computed by Algorithm 1 in $\mathcal{N}_u(\mathcal{G})$ with parameter θ. Let $e = (u, v_i, t_i)$ be a temporal edge in $\mathcal{N}_u(\mathcal{G})$, and $[t_i, t_e]$ be its lifespan ($t_e \leq t_i + \theta$). Note that t_e is not necessarily equal to $t_i + \theta$, because v_i may be a repeated neighbor of u (see Section 3.2). Recall that by the meta-interval decomposition technique, there must exist an index l such that $\mathcal{MI}_u(l) = (t_{s_l}, t_{e_l})$ and $t_{s_l} = t_i$. If $t_{e_l} < t_e$, there is an index r ($l < r \leq h$) such that $\mathcal{MI}_u(r) = (t_{s_r}, t_{e_r})$ and $t_{s_r} < t_e \leq t_{e_r}$. Otherwise, we let $r = l$.

By the above definition, the lifespan of the temporal edge $e = (u, v_i, t_i)$ (i.e., $[t_i, t_e]$) *hits* every meta-interval in $\mathcal{I} = \{(t_{s_l} - \theta, t_{e_l}), \cdots, (t_{s_r} - \theta, t_{e_r})\}$. Clearly, for node u, the deletion (or insertion) of a temporal edge $e = (u, v_i, t_i)$ only affects the θ-persistent degree of u in each meta-interval in $\mathcal{I}$. Specifically, we have the following results.

LEMMA 6. *After deleting a temporal edge* $e = (u, v_i, t_i)$ *from* $\mathcal{N}_u(\mathcal{G})$*, we decrease* $\mathcal{D}_u(j)$ *by* 1 *for all* $j = l, \cdots, r$*, and keep* $\mathcal{D}_u(j)$*, for all* $j \notin [l,r]$*, unchanged. Based on the updated* $\mathcal{D}_u$ *array, we have* $f(u,\theta,k,\mathcal{G}\backslash\{e\}) = \sum_{\mathcal{D}_u(i)\geqslant k}(t_{e_i} - t_{s_i}) + \theta$*.*

LEMMA 7. *After inserting a temporal edge* $e = (u, v_i, t_i)$ *back into* $\mathcal{N}_u(\mathcal{G})\backslash\{e\}$*, we increase* $\mathcal{D}_u(j)$ *by* 1 *for all* $j = l, \cdots, r$ *and keep* $\mathcal{D}_u(j)$*, for all* $j \notin [l,r]$*, unchanged. Based on the updated* $\mathcal{D}_u$ *array, we have* $f(u,\theta,k,\mathcal{G}) = \sum_{\mathcal{D}_u(i)\geqslant k}(t_{e_i} - t_{s_i}) + \theta$*.*

Lemmas 6 and 7 show that we only need to update the array $\mathcal{D}_u$ when we delete an edge from $\mathcal{N}_u(\mathcal{G})$ or add back an edge e into $\mathcal{N}_u(\mathcal{G})\backslash\{e\}$. It is important to note that we do not destroy the original meta-interval structure of u computed in $\mathcal{N}_u(\mathcal{G})$ when updating u's degree persistence. Moreover, since the lifespan of each temporal edge e (i.e., $[t_i, t_e]$) is no larger than θ by our definition, the number of elements in $\mathcal{D}_u$ that are needed to be updated is bounded by θ (because $[t_i, t_e]$ hits at most θ meta-intervals). As a consequence, we can update the degree persistence in $O(\theta)$ time after obtaining the meta-interval decomposition. The following example illustrates the correctness of Lemmas 6 and 7.

EXAMPLE 10. *Reconsider the node* v_1 *in Fig. 1(a). Let* $k = 2$ *and* $\theta = 3$*. By the meta-interval decomposition, we obtain* $\mathcal{D}_{v_1} = \{1,2,2,1,1,2,1\}$ *and* $\mathcal{MI}_{v_1} = \{(1,2),(2,3),(3,5),(5,6),(6,7),(7,9),(9,10)\}$*. Suppose that we delete the temporal edge* $e = (v_1, v_2, 1)$*. Since the lifespan of* e *is* $[1,3]$ *and* $(t_{s_1}, t_{e_1}) = (1,2)$*, we have* $l = 1$*. Since* $t_{e_l} = 2 < t_e$ *and* $t_{s_2} < t_e \leq t_{e_2}$*, we have* $r = 2$*. Therefore, by Lemma 6, we decrease both* $\mathcal{D}_{v_1}(1)$ *and* $\mathcal{D}_{v_1}(2)$ *by* 1*. The degree persistence is updated by* 7*, because there are two meta-intervals with* θ*-persistent degree no less than* k *(intervals* $(3,5)$ *and* $(7,9)$*). Note that after deleing* e*, the maximal* (θ,k)*-persistent-degree intervals of* v_1 *are* $[0,5]$ *and* $[4,9]$ *by Definition 4. Thus, our result is consistent with the result obtained by definition. On the other hand, if we add back* e *into* $\mathcal{N}_{v_1}(\mathcal{G}\backslash\{e\})$*, we increase both* $\mathcal{D}_{v_1}(1)$ *and* $\mathcal{D}_{v_1}(2)$ *by* 1 *based on Lemma 7. We can verify that the degree persistence of* v_1 *is updated by* 8*, which is consistent with the result obtained by Definition 4.*

3.5 The TGR algorithm

Armed with the results shown in the previous sections, we are now ready to present the TGR algorithm. The TGR algorithm is outlined in Algorithm 2. Specifically, Algorithm 2 first invokes Algorithm 1 to compute the meta-interval decomposition for every node $u \in \mathcal{V}$ (line 3). Then, the algorithm calculates the degree persistence based on the meta-interval decomposition, and pushes all the nodes with degree persistence less than τ into the queue $\mathcal{Q}$ (lines 4-6). Subsequently, the algorithm iteratively removes the nodes with degree persistence less than τ (lines 7-19). In each iteration, the algorithm pops a node from $\mathcal{Q}$ (line 8), and traverses its temporal neighborhood $\mathcal{N}_u(\mathcal{G})$. For each temporal edge $e = (u, v, t) \in \mathcal{N}_u(\mathcal{G})$, the algorithm updates the degree persistence of v after deleing e based on Lemma 6 (lines 10-19). If the updated degree persistence of a node is smaller than τ, the algorithm pushes it into the queue $\mathcal{Q}$ (line 18). The algorithm terminates when $\mathcal{Q}$ is empty. When the algorithm terminates, all the remaining nodes in $\mathcal{G}$ have degree persistence no less than τ. The complexity of Algorithm 2 is analyzed in Theorem 7.

THEOREM 7. *The time and space complexity of Algorithm 2 is* $O(\theta m)$ *and* $O(m)$ *respectively.*

Algorithm 2 TGR $(\mathcal{G}, \theta, k, \tau)$

Input: $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, θ, k, and τ
Output: The reduced temporal graph $\tilde{\mathcal{C}}$

$\mathcal{Q} \leftarrow \emptyset$; $flag(u) \leftarrow 1$ for all $u \in \mathcal{V}$;
for all $u \in \mathcal{V}$ **do**
  $\{\mathcal{D}_u, \mathcal{MI}_u\} \leftarrow$ Meta-interval-Decomposition $(u, \mathcal{N}_u, \theta)$;
  Compute $f(u, \theta, k, \mathcal{G})$ (by Theorems 5 and 6); $\tilde{d}_u \leftarrow f(u, \theta, k, \mathcal{G})$;
  if $\tilde{d}_u < \tau$ **then**
    $\mathcal{Q}.push(u)$; $flag(u) \leftarrow 0$;
while $\mathcal{Q} \neq \emptyset$ **do**
  $u \leftarrow \mathcal{Q}.pop()$;
  for all $(u, v, t) \in \mathcal{N}_u(\mathcal{G})$ **do**
    Let $[t, t_e]$ be the lifespan of the temporal edge (u, v, t);
    Let $\mathcal{MI}_v = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$;
    Suppose $t_{s_i} = t$; $j \leftarrow i$;
    while $j \leq h$ and $t_{s_j} < t_e$ **do**
      $\mathcal{D}_v(j) \leftarrow \mathcal{D}_v(j) - 1$;
      if $\mathcal{D}_v(j) + 1 \geq k$ and $\mathcal{D}_v(j) < k$ **then**
        $\tilde{d}_v \leftarrow \tilde{d}_v - (t_{e_j} - t_{s_j})$; {// Update the degree persistence}
        if $\tilde{d}_v + (t_{e_j} - t_{s_j}) \geq \tau$ and $\tilde{d}_v < \tau$ **then**
          $\mathcal{Q}.push(v)$; $flag(v) \leftarrow 0$;
      $j \leftarrow j + 1$;
return $\tilde{\mathcal{C}} \leftarrow$ Subgraphs induced by the nodes with $flag(u) = 1$;

4. ALGORITHM FOR PC SEARCH

In this section, we propose a tractable algorithm for the PC search problem. Our algorithm includes two stages. First, we invoke Algorithm 2 to reduce the temporal graph. Then, in the reduced temporal graph, we propose an efficient branch and bound algorithm with several powerful pruning rules to compute the largest (θ, τ)-persistent k-core. Below, we first propose three key sub-algorithms which will be frequently invoked in the branch and bound algorithm.

4.1 Three key sub-algorithms

Compute common meta-intervals. Here we propose an algorithm to compute the common meta-intervals for a set of nodes which will be utilized to evaluate the core persistence. The algorithm is outlined in Algorithm 3. Given a set of nodes $\mathcal{R}$, Algorithm 3 outputs the common meta-intervals such that in each of those meta-intervals every node in $\mathcal{R}$ has a θ-persistent degree no less than k. Algorithm 3 first initializes a common meta-interval array $\mathcal{MIR} = \{(-\infty, +\infty)\}$ for an empty node set (line 1). Then, the algorithm iteratively traverses a node $u \in \mathcal{R}$, and takes the interval-intersection between the common meta-interval array $\mathcal{MIR}$ and the meta-interval array of u (lines 2-19). In each iteration, the algorithm updates $\mathcal{MIR}$ by the resulting meta-intervals (line 19). Note that for each node $u \in \mathcal{R}$, we only need to consider the meta-intervals $\mathcal{MI}_u(i)$ for all $i = 1, \cdots, h$ such that $\mathcal{D}_u(i) \geq k$ (line 7). For convenience, we refer to those meta-intervals as *active* meta-intervals. This is because only the active meta-intervals will be used to compute the core persistence. Algorithm 3 also outputs len which is equal to the total length of the common meta-intervals plus θ (line 20). Note that if $\mathcal{R}$ forms a connected k-core and all $\mathcal{D}_u$ for $u \in \mathcal{R}$ have been updated w.r.t. $\mathcal{R}$, we can easily obtain that $F(\theta, k, \mathcal{R}) = len + \theta$ (similar to the results of Theorem 5).

EXAMPLE 11. *Reconsider the temporal graph $\mathcal{G}$ in Fig. 1. Assume that $k = 2$, $\theta = 3$, and $\mathcal{R} = \{v_1, v_2, v_3\}$. Clearly, $\mathcal{R}$ is a connected 2-core. In the entire temporal graph $\mathcal{G}$, v_1 has three active meta-intervals $(2, 3)$, $(3, 5)$ and $(7, 9)$, in which the corresponding $\mathcal{D}_u(i) \geq 2$. However, if we only consider the nodes in $\mathcal{R}$, we can see that v_1 has two active meta-intervals $(2, 3)$ and $(3, 5)$. Similarly, we can derive that v_2 and v_3 has an active meta-interval $(4, 6)$ and $(4, 5)$ respectively. By Algorithm 3, we are able to obtain that the common meta-interval of $\mathcal{R}$ is $(4, 5)$, and the total length is 1. Therefore, the core persistence is 4, which is consistent with the result shown in Example 1.*

Algorithm 3 Meta-Interval-Intersection $(\mathcal{R}, k)$

1: Let $\mathcal{MIR} \leftarrow \{(-\infty, +\infty)\}$; $len \leftarrow +\infty$;
2: **for all** $u \in \mathcal{R}$ **do**
3: Let $h \leftarrow |\mathcal{MI}_u|$, and $\mathcal{MI}_u = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$;
4: Let $p \leftarrow |\mathcal{MIR}|$, and $\mathcal{MIR} = \{(t_{l_1}, t_{r_1}), \cdots, (t_{l_p}, t_{r_p})\}$;
5: $\mathcal{MIT} \leftarrow \emptyset$; $tlen \leftarrow 0$; $j \leftarrow 1$;
6: **for** $i = 1$ to h **do**
7: **if** $\mathcal{D}_u(i) \geq k$ **then**
8: **while** $j \leq p$ and $t_{r_j} \leq t_{s_i}$ **do**
9: $j \leftarrow j + 1$;
10: **while** $j \leq p$ and $t_{r_j} \leq t_{e_i}$ **do**
11: $l \leftarrow \max\{t_{l_j}, t_{s_i}\}$; $r \leftarrow t_{r_j}$;
12: **if** $l < r$ **then**
13: $\mathcal{MIT} \leftarrow \mathcal{MIT} \cup \{(l, r)\}$; $tlen \leftarrow tlen + (r - l)$;
14: $j \leftarrow j + 1$čž
15: **if** $j \leq p$ and $t_{l_j} \leq t_{e_i}$ **then**
16: $l \leftarrow \max\{t_{l_j}, t_{s_i}\}$; $r \leftarrow t_{e_i}$;
17: **if** $l < r$ **then**
18: $\mathcal{MIT} \leftarrow \mathcal{MIT} \cup \{(l, r)\}$; $tlen \leftarrow tlen + (r - l)$;
19: $\mathcal{MIR} \leftarrow \mathcal{MIT}$; $len \leftarrow tlen$;
20: **return** $\{\mathcal{MIR}, len + \theta\}$;

Let N_u be the number of meta-intervals of u with $\mathcal{D}_u(i) \geq k$ for $i = 1, \cdots, h$ (i.e., N_u denotes the number of active meta-intervals.). Then, we analyze the time complexity of Algorithm 3 in Theorem 8.

THEOREM 8. *Let $N_R = \sum_{u \in \{\mathcal{R}\}} N_u$. The time complexity of Algorithm 3 is $O(|\mathcal{R}| N_R)$.*

Remove nodes. We present an algorithm to maintain the degree persistence of the nodes after deleting a node u, because our branch and bound algorithm frequently invokes the node-removal procedure to prune the search space. The algorithm is described in Algorithm 4. In Algorithm 4, the parameter u is the node that we want to delete, and the parameter $\mathcal{S}$ denotes a set of nodes that cannot be deleted. After removing u, we only need to maintain the degree persistence of the neighbors of u. Thus, the algorithm iteratively traverses a temporal edge in $\mathcal{N}_u(\mathcal{G})$. In each iteration, the algorithm update the degree persistence of u's neighbors based on the result shown in Lemma 6 (lines 3-14). The algorithm makes use of a set $\mathcal{R}$ to record the neighbors whose updated degree persistence is smaller than τ (lines 10-11). Since the nodes in $\mathcal{R}$ cannot be contained in the (θ, τ)-persistent k-core, all nodes in $\mathcal{R}$ can be deleted. If $\mathcal{R}$ consists of a node in $\mathcal{S}$, the algorithm returns false, indicating that we cannot delete node u (lines 12-13). The time complexity of Algorithm 4 is shown in the following theorem.

THEOREM 9. *The time complexity of Algorithm 4 is $O(\theta |\mathcal{N}_u(\mathcal{G})|)$.*

Add nodes. Similarly, we present Algorithm 5 to maintain the degree persistence of the nodes after adding a node u. This is because after deleting the nodes by the branch and bound algorithm, we need to add back the removed nodes to recover the search state (i.e., the backtracking procedure). Algorithm 5 is based on the results shown in Lemma 7. Since the general procedure of the algorithm is very similar to Algorithm 4, we omit the details. The time complexity of Algorithm 5 is $O(\theta |\mathcal{N}_u(\mathcal{G})|)$.

4.2 The branch and bound framework

Equipped with the above three sub-algorithms, we propose the branch and bound algorithm, as well as the PC search algorithm in

Algorithm 4 Remove-Node (u, $\mathcal{S}$, k, τ)

$flag \leftarrow 1; \mathcal{R} \leftarrow \emptyset$;
for all $(u, v, t) \in \mathcal{N}_u(\mathcal{G})$ **do**
  Let $[t, t_e]$ be the lifespan of (u, v, t);
  Let $\mathcal{MI}_v = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$;
  Suppose $t_{s_i} = t; j \leftarrow i$;
  while $j \leq h$ and $t_{s_j} < t_e$ **do**
    $\mathcal{D}_v(j) \leftarrow \mathcal{D}_v(j) - 1$;
    if $\mathcal{D}_v(j) + 1 \geq k$ and $\mathcal{D}_v(j) < k$ **then**
      $\tilde{d}_v \leftarrow \tilde{d}_v - (t_{e_j} - t_{s_j})$; {// Update the degree persistence}
      if $\tilde{d}_v + (t_{e_j} - t_{s_j}) \geq \tau$ and $\tilde{d}_v < \tau$ **then**
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{v\}$;
        if $v \in \mathcal{S}$ **then**
          $flag \leftarrow 0$;
    $j \leftarrow j + 1$;
return $\{\mathcal{R}, flag\}$;

Algorithm 5 Add-Node (u, k)

for all $(u, v, t) \in \mathcal{N}_u(\mathcal{G})$ **do**
  Let $[t, t_e]$ be the lifespan of (u, v, t);
  Let $\mathcal{MI}_v = \{(t_{s_1}, t_{e_1}), \cdots, (t_{s_h}, t_{e_h})\}$;
  Suppose $t_{s_i} = t; j \leftarrow i$;
  while $j \leq h$ and $t_{s_j} < t_e$ **do**
    $\mathcal{D}_v(j) \leftarrow \mathcal{D}_v(j) + 1$;
    if $\mathcal{D}_v(j) \geq k$ and $\mathcal{D}_v(j) - 1 < k$ **then**
      $\tilde{d}_v \leftarrow \tilde{d}_v + (t_{e_j} - t_{s_j})$; {// Update the degree persistence}
    $j \leftarrow j + 1$;

Algorithm 6 and Algorithm 7 respectively. In Algorithm 7, we first call Algorithm 2 to reduce the temporal graph (line 2), and then invokes Algorithm 6 to compute the maximum (θ, τ)-persistent k-core. Below, we details Algorithm 6.

The key idea of Algorithm 6. Let $\tilde{\mathcal{C}}$ be the input graph (i.e., the reduced temporal graph) of the algorithm. First, we compute all the maximal connected components of $\tilde{\mathcal{C}}$ (line 2). For each component $\mathcal{C}$, we invoke Algorithm 3 to compute the total length of the common meta-intervals of the nodes in $\mathcal{C}$, denoted by len (line 6). If $len + \theta \geq \tau$, $\mathcal{C}$ must be a (θ, τ)-persistent k-core by Definition 3 (line 7). Otherwise, we divide the search space into two disjoint subspaces by randomly picking a node u in $\mathcal{C}$ (line 10): (1) the subspace of excluding u, and (2) the subspace of including u. Clearly, any (θ, τ)-persistent k-core must be contained in one of these two subspaces. We make use of a set $\mathcal{S}$ to maintain all the included nodes.

Subspace-I: excluding u. In this subspace, the resulting (θ, τ)-persistent k-cores cannot contain u. Thus, we invoke Algorithm 4 to delete u (line 15). Since removing u may trigger the deletions of the other nodes, Algorithm 4 returns all these deleted nodes $\mathcal{R}$ and an indicator variable $flag$. If $flag = 1$, we remove all the nodes in $\mathcal{R}$ from $\mathcal{C}$, and then recursively invoke the branch and bound algorithm in the reduced subgraph $\mathcal{C} \backslash \mathcal{R}$ (line 17). Otherwise, we can safely prune this subspace, because we fail to delete u, implying that there is no (θ, τ)-persistent k-core in this subspace (line 16). After this step, we backtrack to the search space before removing u. This can be done by invoking Algorithm 5 to add back the deleted nodes (lines 18-20).

Subspace-II: including u. In this subspace, we invoke the branch and bound algorithm subject to the constraint that the resulting (θ, τ)-persistent k-cores must contain u. After that, we backtrack to the search space before including u, which can be done by removing u from $\mathcal{S}$ (lines 21-40).

The algorithm uses a variable $size$ to record the largest size of the (θ, τ)-persistent k-cores computed so far. If $|\mathcal{C}| < size$, we

Algorithm 6 Branch-Bound ($\tilde{\mathcal{C}}$, $\mathcal{S}$, k, τ)

1: **if** $|\tilde{\mathcal{C}}| < size$ **then return**;
2: $\mathcal{CM} \leftarrow$ All-Components ($\tilde{\mathcal{C}}$); {// compute the connected components}
3: **for all** $\mathcal{C} \in \mathcal{CM}$ **do**
4: **if** $\mathcal{S} \subseteq \mathcal{C}$ **then**
5: **if** $|\mathcal{C}| < size$ **then return**;
6: $\{\mathcal{MIC}, lc\} \leftarrow$ Meta-Interval-Intersection($\mathcal{C}$, k);
7: **if** $lc \geq \tau$ **then** $\mathcal{R}^* \leftarrow \mathcal{C}$; $size \leftarrow |\mathcal{C}|$; **return**;
8: $\{\mathcal{MIS}, ls\} \leftarrow$ Meta-Interval-Intersection($\mathcal{S}$, k);
9: **if** $ls \geq \tau$ and $|\mathcal{S}| < |\mathcal{C}|$ **then**
10: Randomly pick a node $u \in \mathcal{C} \backslash \mathcal{S}$;
11: $f_u \leftarrow \tilde{d}_u$; $\tilde{d}_u \leftarrow 0$; {// f_u records the degree persistence}
12: $\mathcal{Q} \leftarrow \emptyset$; $\mathcal{Q}.push(u)$; $\mathcal{A} \leftarrow \emptyset$;
13: **while** $\mathcal{Q} \neq \emptyset$ **do**
14: $u \leftarrow \mathcal{Q}.pop()$; $\mathcal{A} \leftarrow \mathcal{A} \cup \{u\}$;
15: $\{\mathcal{H}, flag\} \leftarrow$ Remove-Node (u, $\mathcal{S}$, k, τ);$\mathcal{Q} \leftarrow \mathcal{Q} \cup \mathcal{H}$;
16: **if** $flag = 0$ **then break**;
17: **if** $flag = 1$ **then** Branch-Bound ($\mathcal{C} \backslash \mathcal{A}$, S, k, τ);
18: $\tilde{d}_u \leftarrow f_u$;
19: **for all** $v \in \mathcal{A}$ **do**
20: Add-Node (v, k);
21: $\mathcal{S} \leftarrow \mathcal{S} \cup \{u\}$; {// the case of including u}
22: $\{\mathcal{MIS}, ls\} \leftarrow$ Meta-Interval-Intersection($\mathcal{S}$, k);
23: **if** $ls \geq \tau$ **then**
24: $\mathcal{B} \leftarrow \emptyset$; $\mathcal{Q} \leftarrow \emptyset$;
25: **for all** $v \in \mathcal{C} \backslash \mathcal{S}$ **do**
26: $\{\mathcal{MIV}, lv\} \leftarrow$ Meta-Interval-Intersection($\mathcal{S} \cup \{v\}$, k);
27: **if** $lv < \tau$ **then**
28: $f_v \leftarrow \tilde{d}_v$; $\tilde{d}_v \leftarrow 0$;
29: $\mathcal{B} \leftarrow \mathcal{B} \cup \{v\}$;
30: $\mathcal{Q} \leftarrow \mathcal{B}$; $\mathcal{A} \leftarrow \emptyset$;
31: **while** $\mathcal{Q} \neq \emptyset$ **do**
32: $v \leftarrow \mathcal{Q}.pop()$; $\mathcal{A} \leftarrow \mathcal{A} \cup \{v\}$;
33: $\{\mathcal{H}, flag\} \leftarrow$ Remove-Node (u, $\mathcal{S}$, k, τ);$\mathcal{Q} \leftarrow \mathcal{Q} \cup \mathcal{H}$;
34: **if** $flag = 0$ **then break**;
35: **if** $flag = 1$ **then** Branch-Bound ($\mathcal{C} \backslash \mathcal{A}$, S, k, τ);
36: **for all** $v \in \mathcal{B}$ **do**
37: $\tilde{d}_v \leftarrow f_v$;
38: **for all** $v \in \mathcal{A}$ **do**
39: Add-Node (v, k);
40: $\mathcal{S} \leftarrow \mathcal{S} \backslash \{u\}$;

Algorithm 7 PC ($\mathcal{G}$, θ, k, τ)

Input: $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, θ, k, and τ
Output: The maximum (θ, τ)-persistent k-core

1: $\mathcal{R}^* \leftarrow \emptyset$; $size \leftarrow 0$; {// global variables}
2: $\tilde{\mathcal{C}} \leftarrow$ TGR ($\mathcal{G}$, θ, k, τ);
3: Branch-Bound($\tilde{\mathcal{C}}$, $\emptyset$, k, τ);
4: **return** $\mathcal{R}^*$;

can prune this search space, because the (θ, τ)-persistent k-cores contained in this space must be smaller than $size$ (line 5). Below, we propose several effective pruning rules to further reduce the search space.

Upper-bound pruning. Let ls be the total length of the common meta-intervals of the nodes in $\mathcal{S}$ calculated by Algorithm 3. Since the resulting (θ, τ)-persistent k-cores must contain $\mathcal{S}$, $ls + \theta$ is an upper bound of the core persistence of the (θ, τ)-persistent k-core. For a component $\mathcal{C}$ and the included-node set $\mathcal{S}$, we can first compute ls for $\mathcal{S}$. If $ls < \tau$, we can prune the current search space, because the core persistence of the (θ, τ)-persistent k-cores in this space must be smaller than τ (lines 8-9 in Algorithm 6).

Excluding-node pruning. Consider the case of excluding a node u. After invoking Algorithm 4, if $\mathcal{R} \neq \emptyset$ and $flag = 1$, we may further delete the node whose degree persistence is smaller than τ when deleting $\mathcal{R}$. This is because the deletion of nodes in $\mathcal{R}$ may trigger the degree persistence of the neighbors of the nodes in $\mathcal{R}$ smaller than τ. As a result, we can iteratively remove the nodes until all the nodes with degree persistence no less than τ (similar to the TGR procedure). To this end, we create a queue

$\mathcal{Q}$ that initially includes all the nodes in $\mathcal{R}$. Then, we pop a node v from $\mathcal{Q}$, and invoke Algorithm 4 to delete v and also to update the degree persistence of v's neighbors. Suppose that Algorithm 4 returns $\mathcal{R}'$ and $flag'$ in an iteration. If $flag' = 0$, we are able to prune this subspace, because we cannot delete u in this case (line 16). Otherwise, we add all nodes in $\mathcal{R}'$ into $\mathcal{Q}$. Then, we iteratively perform the same procedure until $\mathcal{Q} = \emptyset$ (lines 12-16). This pruning rule can largely reduce the number of nodes of the input component, thus substantially reduce the search time as verified in our experiments.

Including-node pruning. Consider the case of including a node u. Assume that the current input component is $\mathcal{C}$ and the included-node set is $\mathcal{S}$. We have several pruning tricks in this case. First, we can apply the upper-bound pruning rule to prune the unpromising search space. Specifically, we compute the total length of the common meta-intervals of the nodes in $\mathcal{S} \cup \{u\}$, and then we make use of the upper-bound pruning rule to prune the search space (lines 21-23). Second, for each node $v \in \mathcal{C}$, we compute the total length of the common meta-intervals of the nodes in $\mathcal{S} \cup \{u, v\}$, denoted by lv. If $lv + \theta < \tau$, we can conclude that v cannot be contained in any (θ, τ)-persistent k-core that already consists of $\mathcal{S} \cup \{u\}$. We use a set $\mathcal{B}$ to record all those nodes (lines 24-29). Clearly, we can delete all the nodes in $\mathcal{B}$. Note that the deletion of the nodes in $\mathcal{B}$ may result in the other nodes that must also be removed. As a consequence, we can use the same method as used in the excluding-node pruning to prune the unpromising search spaces (lines 30-35). As demonstrated in the experiments, this pruning rule can further reduce the search time significantly.

Since Algorithm 6 does not miss any search space, the results obtained by our algorithm are correct. Below, we analyze the complexity of our algorithm.

Complexity analysis. Since the problem of finding the largest (θ, τ)-persistent k-core is NP-hard, the worst-case time complexity of Algorithm 7 is exponential. More specifically, there are at most $O(2^{\tilde{n}})$ subspaces (because the recursion tree of our algorithm is clearly a binary tree), where $\tilde{n}$ denotes the size of the largest connected component of the reduced temporal graph. In each subspace, the algorithm takes at most $O(\theta m)$ time to delete and add nodes, and $O(\tilde{n}N)$ time to compute the common meta-intervals, where N denotes the maximum number of active meta-intervals in the reduced temporal graph. Therefore, the worst-case time complexity of Algorithm 7 is $O(2^{\tilde{n}}(\tilde{n}N + \theta m)n_c)$, where n_c denotes the number of connected component of the reduced temporal graph. Since $\tilde{n}$ is typically not very large and the proposed pruning rules are very effective, the proposed algorithm is tractable in many real-world large-scale temporal graphs. In the experiments, we will show that our algorithm is scalable to handle the temporal graph with more than one million nodes and ten millions edges.

Discussions. The proposed branch and bound framework can also be used for enumerating all (θ, τ)-persistent k-cores. To this end, we remove the size constraint in Algorithm 6, and add a constraint of $|\mathcal{C}| = |\mathcal{S}|$ for maximal property testing. Specifically, in Algorithm 6, we delete lines 1 and 5. In line 7, we add a constraint $|\mathcal{C}| = |\mathcal{S}|$. If $|\mathcal{C}| = |\mathcal{S}|$ and $lc \geq \tau$, we output $\mathcal{C}$ as a (θ, τ)-persistent k-core. The reasons are as follows. First, $\mathcal{C}$ is connected and the core persistence of $\mathcal{C}$ is no less than τ. Second, since $|\mathcal{C}| = |\mathcal{S}|$, there is no candidate node that can be added into $\mathcal{S}$, thus $\mathcal{S} = \mathcal{C}$ satisfies the maximal property. As a result, $\mathcal{C}$ is a (θ, τ)-persistent k-core.

5. EXPERIMENTS

Dataset	n	m
Chess	7,301	65,053
Linux	63,399	1,028,233
Enron	87,273	1,134,990
DBLP	1,570,522	11,596,310

Table 1: Datasets

Parameters	θ	k	τ
Chess	7, $[5, 9]$	4, $[3, 6]$	16, $[12, 20]$
Linux	7, $[3, 11]$	10, $[6, 14]$	30, $[22, 38]$
Enron	6, $[4, 8]$	4, $[3, 7]$	22, $[18, 26]$
DBLP	4, $[2, 6]$	8, $[4, 12]$	18, $[14, 22]$

Table 2: Parameters (default value and range)

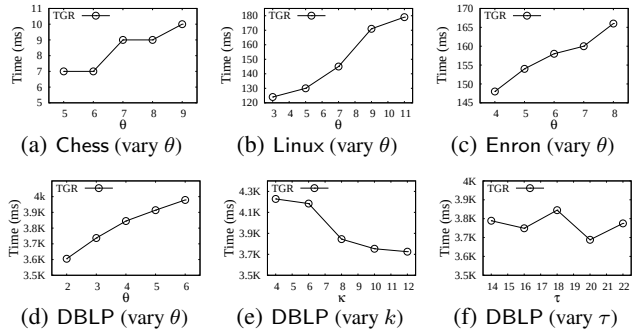
We conduct comprehensive experiments to evaluate the efficiency and effectiveness of the proposed algorithms. We implement the TGR algorithm (Algorithm 2) to prune the temporal graph. For the PC search problem, we implement four various algorithms: Basic, BB+, BB++, and BB-All. All these algorithms first invoke TGR to reduce the temporal graph, and then call Algorithm 6 to find the maximum persistent k-core. The differences among these algorithms are the pruning rules used in Algorithm 6. Specifically, Basic does not use any pruning rule presented in Section 4.2. We make use of this algorithm as the baseline for efficiency testings, because no existing algorithm can be used to find persistent k-cores. BB+ denotes the Basic algorithm with upper-bound pruning. BB++ is the BB+ algorithm with excluding-node pruning, and BB-All denotes BB++ with including-node pruning. All algorithms are implemented in C++. All experiments are conducted on a computer with two 3.46GHz Xeon CPUs and 64GB memory running Red Hat Enterprise Linux 6.4.

Datasets. We use four different types of real-world temporal networks in the experiments. The detailed statistics of our datasets are summarized in Table 1. All the datasets are downloaded from (`http://konect.uni-koblenz.de`). Chess is a temporal network in which a temporal edge (u, v, t) represents a chess game between the players u and v at time t. Linux is a temporal communication network of the Linux kernel mailing list, where a temporal edge (u, v, t) denotes a reply from a user u to v at time t. Enron is a temporal email network between employees of Enron between 1999 and 2003. DBLP is a temporal collaboration network of authors recorded in DBLP.

Parameters. There are three parameters in our community models which are θ, k and τ. Note that if θ is too large, the persistent constraint of our models will be very loose, and thus the resulting communities are not the qualified persistent communities. On the other hand, if θ is too small, there will be no persistent community. Similarly, if k and/or τ are too large, there will be no answer, while if k and/or τ are too small, the obtained communities are not cohesive and are also not persistent. We will study how these parameters affect the community size in the following experiments. Table 2 shows the ranges and default values of the parameters used in the experiments. Note that since the time scales of the datasets are diverse, the parameter settings are also different for various datasets. When varying a certain parameter, the values for all the other parameters are set to their default values.

5.1 Efficiency testings

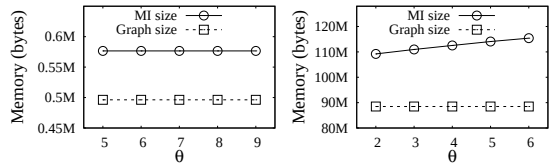
Efficiency of the TGR algorithm. In this experiment, we test the efficiency of the TGR algorithm (Algorithm 2) with varying θ, k, and τ. The results of varying θ in all the datasets are shown



(a) Chess (vary θ) (b) Linux (vary θ) (c) Enron (vary θ)

(d) DBLP (vary θ) (e) DBLP (vary k) (f) DBLP (vary τ)

Figure 2: Efficiency of the TGR algorithm



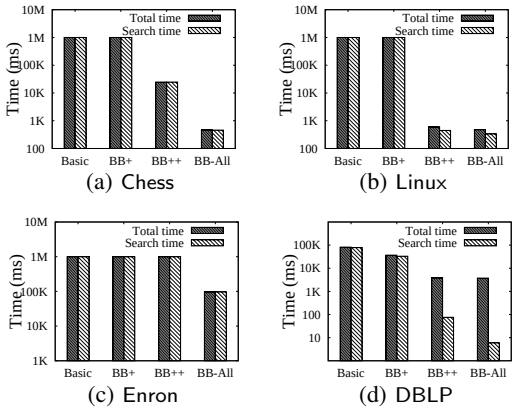
(a) Chess (vary θ) (b) DBLP (vary θ)

Figure 3: Memory overhead of the meta-interval structure

in Figs. 2(a-d). As can be seen, the running time of the algorithm increases with increasing θ in all datasets. This is because the time complexity of the TGR algorithm is $O(\theta m)$, which is linearly dependent on θ. For the results of varying k and τ, we only report the results in the DBLP dataset in Figs. 2(e-f), and similar results can also be observed in the other datasets. From Fig. 2(e), we can see that the running time of TGR decreases when k increases. This is because for a large k, the algorithm can quickly remove a large number of unpromising nodes, and thus significantly reduces the running time of the iterative-refinement procedure. As observed in Fig. 2(f), the efficiency of TGR seems not very sensitive w.r.t. τ. In DBLP, the fluctuation of the running time of our algorithm is around 0.1 seconds. Additionally, we can see that the TGR algorithm is very efficient in all testings. For example, in DBLP, TGR takes less than 4 seconds under the default parameters setting. These results confirm the complexity analysis shown in Section 3.5.

Memory overhead of the meta-interval structure. In this experiment, we evaluate the memory overhead of the proposed meta-interval structure, because the space costs of all the proposed algorithms are dominated by this structure. Fig. 3 depicts the size of the meta-interval structure with varying θ in the Chess and DBLP datasets, and similar results can be observed in the other datasets. Note that the size of the meta-interval structure is independent on the parameters k and τ. As can be seen, the size of our structure is robust with varying θ, because the number of meta-intervals is typically not very sensitive w.r.t. θ. Also, we can observe that the size of the meta-interval structure is slightly larger than the graph size, this is because the space complexity of our structure is linear w.r.t. the graph size. For instance, in DBLP, the graph consumes 90MB memory, while the meta-interval structure uses less than 120MB memory. These results confirm the space complexity analysis in Section 3.2.

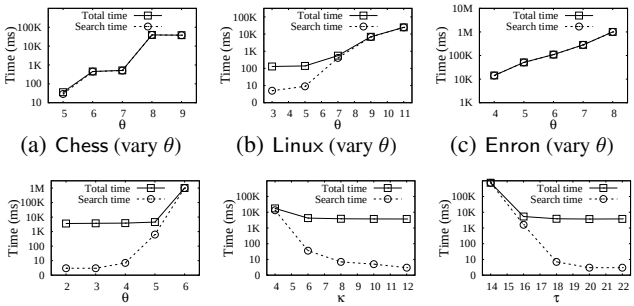
Comparison of different PC search algorithms. In this experiment, we compare the efficiency of different PC search algorithms. The results are reported in Fig. 4. For all the algorithms, the search time is the time spent in the branch and bound procedure (excluding the time taken by the TGR procedure), while the total time denotes total running time of the algorithm. Since some of the PC search algorithms may not be tractable, we limit the maximal running time by 1000 seconds for all algorithms. As can be seen, both



(a) Chess (b) Linux

(c) Enron (d) DBLP

Figure 4: Comparison of different PC search algorithms



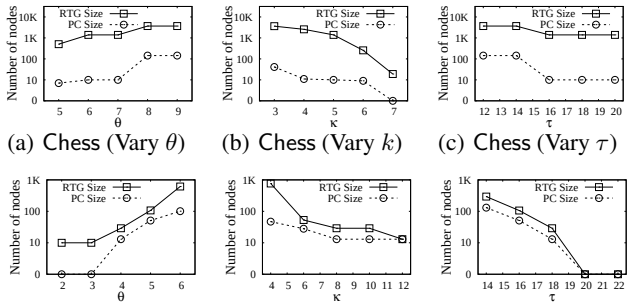
(a) Chess (vary θ) (b) Linux (vary θ) (c) Enron (vary θ)

(d) DBLP (vary θ) (e) DBLP (vary k) (f) DBLP (vary τ)

Figure 5: Efficiency of BB-All with varying parameters

Basic and BB+ are very costly, which are intractable in the Chess, Linux, and Enron datasets. BB++ is very efficient in the Chess, Linux, and DBLP datasets. It is at least two orders of magnitude faster than both Basic and BB+. The best algorithm is the BB-All algorithm, which is very efficient in all datasets. Moreover, we can see that the search time of BB-All is generally one order of magnitude faster than BB++. For example, in DBLP, BB-All takes only 6ms, whereas BB++ consumes 76ms. These results indicate that both the excluding-node pruning and including-node pruning are very effective to prune the search space.

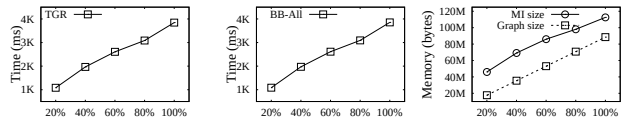
Efficiency of BB-All with varying parameters. Since BB-All is significantly more efficient than the other algorithms, in this experiment we study how the parameters affect the performance of the BB-All algorithm. Figs. 5(a-d) depict the results with varying θ in all the datasets. As shown in Figs. 5(a-d), both the total time and search time increase with increasing θ. This is because there could be a large number of (θ, τ)-persistent k-cores for a large θ, and thus the search space will significantly increase with increasing θ. The total time is dominated by the search time in the first three datasets. However, in DBLP, when θ is small, the total time is dominated by the TGR procedure. This is because when θ is small, the size of the reduced temporal graph in DBLP could be very small, and thus the branch and bound procedure will be much faster. Figs. 5(e-f) report the results in DBLP when varying k and τ respectively. Similar results can also be observed in the other datasets. We can observe that both the total time and search time decrease with increasing k or τ. This is because for a large k or τ, the size of the reduced temporal graph could be small, and therefore the branch and bound search algorithm will be very efficient. In most parameter testings, the running time of BB-All is no large than 15 minutes. These results indicate that BB-All is indeed tractable for real-world large temporal graphs.



(a) Chess (Vary θ) (b) Chess (Vary k) (c) Chess (Vary τ)

(d) DBLP (Vary θ) (e) DBLP (Vary k) (f) DBLP (Vary τ)

Figure 6: Result size with varying parameters (RTG denotes the reduced temporal graph)



(a) TGR Alg. (b) BB-All Alg. (c) MI size

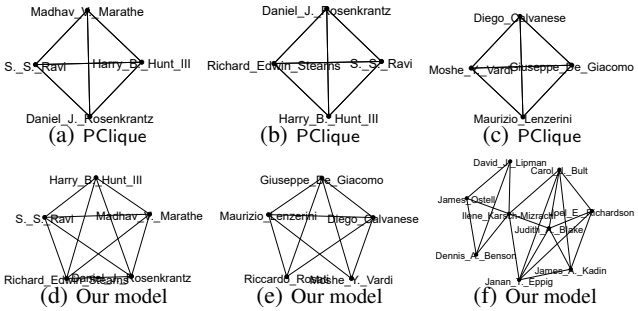
Figure 7: Scalability testing (DBLP)

Result size with varying parameters. Here we evaluate how the parameters affect both the PC (the maximum (θ, τ)-persistent k-core) and the reduced temporal graph (RTG) size. The results in the Chess and DBLP datasets are reported in Fig. 6, similar results can also be observed in other datasets. As can be seen in Fig. 6(a) and Fig. 6(d), when θ increases, the size of both PC and RTG increase. For example, in Chess, when $\theta = 9$ the PC size is 144 and the size of RTG is 3677. Therefore, when θ is large, the community size may be very huge, thus the resulting community could be less useful in practice. On the other hand, if θ is very small, there could be no community. For instance, when $\theta = 2$, there is no (θ, τ)-persistent k-core in DBLP. Consequently, the search time of the proposed algorithms increases with growing θ, which is consistent with our previous results. As desired, in Figs. 6(b), (c), (e), and (f), both the PC size and RTG size decrease when k or τ increases. When k or τ is large, there could be no (θ, τ)-persistent k-core. However, when k or τ is small, the community size could be considerably large, thus the resulting community would be less useful in practice. Moreover, the community may be less cohesive and persistent when k and τ is small. Also, these results further confirm that the search time of our PC search algorithms decreases with increasing k or τ.

Scalability testing. Here we show the scalability of our algorithms. To this end, we generate five temporal subgraphs by taking the first 20%, 40%, 60%, 80%, and 100% temporal edges of the original temporal graph. The results in DBLP are shown in Fig. 7, and similar curves can also be observed in the other datasets. We can see that the TGR algorithm scales linearly w.r.t. the graph size. This is because the time complexity of the TGR algorithm is linear w.r.t. the graph size. We can also observe that the BB-All algorithm also scales very well due to the proposed powerful pruning rules. As desired, the size of meta-interval structure scales linearly with the graph size, because the space complexity of the meta-interval structure is linear.

5.2 Effectiveness testings

We perform extensive case studies to evaluate the effectiveness of our model. We use DBLP and two temporal contact networks Hospital and HighSchool for case studies. Both Hospital and HighSchool are downloaded from (`http://www.sociopatterns.org`). Due to the space limit, here we mainly report the results in DBLP, and the results in Hospital and HighSchool are shown in Appendix A.3. In all case studies, we implement two baselines for comparison: KCore and PClique. KCore is an intuitive baseline which first finds all the persistent edges (with persistence larger than the threshold τ), and then computes the k-core in the temporal subgraph induced by all persistent edges. Note that we can apply the same method as used for computing the core persistence of a temporal subgraph (see Definition 2) to calculate the persistence of an edge, because each edge can be deemed as a 1-core. PClique is a persistent clique model which improves the state-of-the-art temporal clique model [26] to fully capture the persistence of a community. The persistent clique model is detailed in Appendix A.2. In Appendix A.2, we also show that the proposed TGR technique can drastically speed up the persistent clique search algorithm.



(a) PClique (b) PClique (c) PClique

(d) Our model (e) Our model (f) Our model

Figure 8: Comparison of various models ($\theta = 2, k = 3, \tau = 17$)

Case studies in DBLP. We use parameters $\theta = 2$ (i.e., 2 years), $k = 3$, and $\tau = 17$ (i.e., 17 years) in this experiment. More results under the other parameter settings can be found in Appendix A.3. For our model, we obtain 6 communities and the maximum community includes 9 authors. For PClique, we obtain 10 communities and the maximum community is a 4 clique. Fig. 8 reports some of the results obtained by our model and PClique. Note that under the same parameter setting, the community obtained by KCore contains 161 authors (not shown[2]) that come from different research areas including theoretical computer sciences, bioinformatics, artificial intelligence, VLSI design, and so on. In fact, given that $\theta = 2$ and $\tau = 17$, the KCore algorithm can obtain a 2-core including 917 nodes, a 3-core containing 161 nodes, and there does not exist high-order cores (i.e., 3 is the maximum k). Clearly, in DBLP, all the 161 authors found by KCore cannot form a persistent and compact community, as these authors come from diverse research areas. This result indicate that KCore is ineffective to identify persistent communities. Instead, as shown in Fig. 8, both PClique and our model tend to find persistent and compact communities. For example, in Figs. 8(b) and (d), the communities found by both PClique and our model include several famous researchers in the theoretical computer science area who persistently coauthor a paper in the past few decades. Compared to PClique, our model is more effective to detect persistent communities. For example, in Figs. 8(a-b), the results obtained by PClique are redundant, and both of them can only reveal a partial persistent community of the result obtained by our model (Fig. 8(d)). In effect, the five researchers in Fig. 8(d) continuously coauthor papers in the last few decades, thus they should be included in a persistent community. Similar results can also be observed in Figs. 8(c) and (e). In addition, our model can

[2]We do not visualize this community, as it is too large to show in a figure.

find a large persistent community in the bioinformatics area that cannot be found by PClique. The reason could be that the clique constraint in PClique may be too strong, thus ruling out many interesting persistent communities.

6. RELATED WORK

Temporal graph mining. Our work is related to the problem of mining temporal graphs, which has attracted much attention in the database community. For example, Bansal et al. [2] investigated a problem of finding stable keywords clusters in a collection of blog posts for specific temporal intervals. A cluster in [2] is modeled as a bi-connected component of the keyword graph, which could be less cohesive for natural graphs. Wu et al. [28] studied the shortest path problem in temporal graphs. Yang et al. [33] proposed an algorithm to detect frequent changing components in temporal graph. Huang et al. [14] investigated the minimum spanning tree (MST) problem in temporal graphs, where two different types of MSTs were proposed to optimize both the time and cost. Gurukar et al. [11] proposed a model called communication motifs to identify the recurring subgraphs that have similar sequence of information flow. More recently, Wu et al. [30] proposed an efficient algorithm to answer the reachability and time-based path queries in a temporal graph. Yang et al. [32] studied a problem of finding a set of diversified quasi-cliques from a temporal graph. Wu et al. [29] proposed a temporal k-core model based on the counts of temporal edges. Their temporal k-core model does not consider the persistence of the community, thus it cannot be used for detecting persistent communities in a temporal graph. Ma et al. [20] investigated a dense subgraph problem in temporal graphs, in which the temporal edges are associated positive and negative weights. Again, the dense subgraph proposed in their work does not consider the persistence of the community, thus cannot be applied to our problem. To (partially) capture the persistence, Viard et al. [26] proposed a temporal clique model as well as an algorithm using $O(2^n n^2 m^3 + 2^n n^3 m^2)$ time and $O(2^n nm^2)$ space to find all the temporal cliques. Himmel et al. [12] proposed an improved algorithm to enumerate the temporal cliques, but their algorithm is still costly to handle large temporal graphs. Unlike the temporal clique model, our persistent community model is based on the concept of k-core, and the proposed algorithms can be scalable to handle large temporal graphs. In addition, temporal networks were also studied in the physics community and a comprehensive survey can be found in [13].

Cohesive subgraph mining. Our work is also related to cohesive subgraph mining. There are a number of cohesive subgraph models that have been proposed in the literature. Notable cohesive subgraph models include maximal clique [6], k-core [22, 5, 19], k-truss [7, 27, 15], maximal k-edge connected subgraph (MkCS) [34, 4, 1], locally dense subgraph [21], influential community [18], and so on. To mining the maximal cliques from a disk-resident graph, Cheng et al. [6] proposed an I/O-efficient algorithm based on a concept of H-index graph [8]. The same group also proposed several I/O-efficient algorithms to compute the k-core and k-truss efficiently [5, 27]. Huang et al. presented a different k-truss model called k-truss community which can be used to find overlapped communities. To compute the MkCS efficiently, Chang et al. [4] and Akiba et al. [1] proposed two different linear-time algorithms which significantly improve an algorithm proposed by Zhou et al. [34]. More recently, Qin et al. [21] proposed a locally dense subgraph model as well as an efficient algorithm to identify all the disjoint dense subgraphs of a graph. Li et al. [18] introduced an influential community model which can capture both the influence and cohesiveness of the community, and they also proposed an optimal algorithm to compute the influential community efficiently. All the above-mentioned cohesive subgraph models do not consider the temporal information, thus they cannot be used to model persistent communities.

7. CONCLUSION

In this paper, we propose a novel community model called (θ, τ)-persistent k-core to capture the persistence of a community in temporal networks. We prove that the problem of finding the maximum (θ, τ)-persistent k-core is NP-hard. A novel temporal graph reduction technique and a branch and bound algorithm are proposed to solve this problem efficiently. We show that the proposed algorithms can also be used to enumerate all (θ, τ)-persistent k-cores. Comprehensive experiments in real-lief temporal networks demonstrate the efficiency and scalability of our algorithms, as well as the effectiveness of our model.

8. REFERENCES

[1] T. Akiba, Y. Iwata, and Y. Yoshida. Linear-time enumeration of maximal k-edge-connected subgraphs in large networks by random contraction. In *CIKM*, 2013.

[2] N. Bansal, F. Chiang, N. Koudas, and F. W. Tompa. Seeking stable clusters in the blogosphere. In *VLDB*, pages 806–817, 2007.

[3] M. H. Böhlen, J. Gamper, and C. S. Jensen. Multi-dimensional aggregation for temporal data. In *EDBT*, pages 257–275, 2006.

[4] L. Chang, J. X. Yu, L. Qin, X. Lin, C. Liu, and W. Liang. Efficiently computing k-edge connected components via graph decomposition. In *SIGMOD*, 2013.

[5] J. Cheng, Y. Ke, S. Chu, and M. T. Özsu. Efficient core decomposition in massive networks. In *ICDE*, 2011.

[6] J. Cheng, Y. Ke, A. W.-C. Fu, J. X. Yu, and L. Zhu. Finding maximal cliques in massive networks. *ACM Trans. Database Syst.*, 36(4):21, 2011.

[7] J. Cohen. Trusses: Cohesive subgraphs for social network analysis. *Technique report*, 2005.

[8] D. Eppstein and E. S. Spiro. The *h*-index of a graph and its application to dynamic subgraph statistics. In *WADS*, 2009.

[9] J. Fournet and A. Barrat. Contact patterns among high school students. *PLOS ONE*, 9:e107878, 2014.

[10] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[11] S. Gurukar, S. Ranu, and B. Ravindran. COMMIT: A scalable approach to mining communication motifs from dynamic networks. In *SIGMOD*, 2015.

[12] A. Himmel, H. Molter, R. Niedermeier, and M. Sorge. Enumerating maximal cliques in temporal graphs. In *ASONAM*, 2016.

[13] P. Holme and J. Saramaki. Temporal networks. *Physics Reports*, 519:97–125, 2012.

[14] S. Huang, A. W. Fu, and R. Liu. Minimum spanning trees in temporal graphs. In *SIGMOD*, 2015.

[15] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu. Querying k-truss community in large and dynamic graphs. *SIGMOD*, 2014.

[16] D. S. Johnson. The np-completeness column: An ongoing guide. *J. Algorithms*, 8(3):438–448, 1987.

[17] M. Kaufmann, A. A. Manjili, P. Vagenas, P. M. Fischer, D. Kossmann, F. Färber, and N. May. Timeline index: a unified data structure for processing queries on temporal data in SAP HANA. In *SIGMOD*, pages 1173–1184, 2013.

[18] R. Li, L. Qin, J. X. Yu, and R. Mao. Influential community search in large networks. *PVLDB*, 8(5):509–520, 2015.

[19] R. Li, J. X. Yu, and R. Mao. Efficient core maintenance in large dynamic graphs. *IEEE Trans. Knowl. Data Eng.*, 26(10):2453–2465, 2014.

[20] S. Ma, R. Hu, L. Wang, X. Lin, and J. Huai. Fast computation of dense temporal subgraphs. In *ICDE*, 2017.

[21] L. Qin, R. Li, L. Chang, and C. Zhang. Locally densest subgraph discovery. In *KDD*, 2015.

[22] S. B. Seidman. Network structure and minimum degree. *Social networks*, 5(3):269–287, 1983.

[23] M. Sozio and A. Gionis. The community-search problem and how to plan a successful cocktail party. In *KDD*, 2010.

[24] P. Vanhems, A. Barrat, C. Cattuto, J.-F. Pinton, N. Khanafer, C. Regis, B. a Kim, B. Comte, and N. Voirin. Estimating potential infection transmission routes in hospital wards using wearable proximity sensors. *PLoS ONE*, 8:e73970, 2013.

[25] J. Viard, M. Latapy, and C. Magnien. Revealing contact patterns among high-school students using maximal cliques in link streams. In *ASONAM*, 2015.

[26] J. Viard, M. Latapy, and C. Magnien. Computing maximal cliques in link streams. *Theor. Comput. Sci.*, 609:245–252, 2016.

[27] J. Wang and J. Cheng. Truss decomposition in massive networks. *PVLDB*, 5(9):812–823, 2012.
[28] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, and Y. Xu. Path problems in temporal graphs. *PVLDB*, 7(9):721–732, 2014.
[29] H. Wu, J. Cheng, Y. Lu, Y. Ke, Y. Huang, D. Yan, and H. Wu. Core decomposition in large temporal graphs. In *IEEE International Conference on Big Data*, 2015.
[30] H. Wu, Y. Huang, J. Cheng, J. Li, and Y. Ke. Reachability and time-based path queries in temporal graphs. In *ICDE*, 2016.
[31] J. Yang and J. Widom. Incremental computation and maintenance of temporal aggregates. *VLDB J.*, 12(3):262–283, 2003.
[32] Y. Yang, D. Yan, H. Wu, J. Cheng, S. Zhou, and J. C. S. Lui. Diversified temporal subgraph pattern mining. In *KDD*, 2016.
[33] Y. Yang, J. X. Yu, H. Gao, J. Pei, and J. Li. Mining most frequently changing component in evolving graphs. *World Wide Web*, 17(3):351–376, 2014.
[34] R. Zhou, C. Liu, J. X. Yu, W. Liang, B. Chen, and J. Li. Finding maximal k-edge-connected subgraphs from a large graph. In *EDBT*, 2012.

A. APPENDIX

A.1 Missing proofs

LEMMA 1. *Let $[t_{s_1}, t_{e_1}]$ and $[t_{s_2}, t_{e_2}]$ be two overlapped maximal (θ, k)-persistent-core intervals of a temporal graph $\mathcal{G}$, and $t_{e_1} < t_{e_2}$. Then, we have $t_{e_1} - t_{s_2} \leq \theta$.*

PROOF. We prove the lemma by contradiction. Suppose that $t^u_{e_1} - t^u_{s_2} > \theta$. Then, we construct a super-interval $[t^u_{s_1}, t^u_{e_2}]$. In $[t^u_{s_1}, t^u_{e_2}]$, we can easily verify that the temporal graph $\mathcal{G}$ is a k-core in every subinterval of length θ. Since $[t^u_{s_1}, t^u_{e_1}]$ is a subinterval of $[t^u_{s_1}, t^u_{e_2}]$, it contradicts the fact that $[t^u_{s_1}, t^u_{e_1}]$ is a maximal (θ, k)-persistent-core interval. □

THEOREM 1. *The problem of finding the largest (θ, τ)-persistent k-core is NP-hard.*

PROOF. We consider the decision version of Problem 2. Given a temporal graph $\mathcal{G}$, parameters θ, τ, and k, test whether $\mathcal{G}$ contains an induced temporal subgraph with $\eta + 1$ nodes that satisfies the persistent core property. Clearly, the maximum subgraph that meets the persistent core property is the largest (θ, τ)-persistent k-core. To prove the theorem, we construct a reduction from the balanced bipartite subgraph (BBS) problem, which is known to be NP-hard [10, 16]. Given a connected bipartite graph $\mathcal{B} = (\mathcal{X}, \mathcal{Y}, \mathcal{E}_B)$ and an integer η, the BBS problem is to verify whether $\mathcal{B}$ contains a complete bipartite subgraph with η nodes on each side. Based on this, we construct an instance of our problem which consists of a temporal graph $\mathcal{G}$, parameters $\theta = 1$, $\tau = (\eta + 1)$, and $k = 1$. For convenience, we assume that the nodes in $\mathcal{Y}$ are sorted by their identities, denoted by $\mathcal{Y} = \{u_1, u_2, \cdots, u_n\}$. First, we create a node s for the temporal graph, which corresponds to every node $u_i \in \mathcal{Y}$ $(i = 1, \cdots, n)$. For each edge $(v, u_i) \in \mathcal{E}_B$, we create a temporal edge $(s, v, 2 \times i - 1)$. Based on this construction, we obtain a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \mathcal{X} \cup \{s\}$ and $\mathcal{E} = \{(s, v, 2 \times i - 1) | (v, u_i) \in \mathcal{E}_B\}$. Clearly, for each node $u_i \in \mathcal{Y}$, we can obtain a star-shaped subgraph in $\mathcal{G}$ at time $2i - 1$, which is denoted by $\mathcal{S}_i$. Below, we show that the instance of the BBS problem is a Yes-instance if and only if the corresponding instance of our problem is a Yes-instance.

First, suppose that $\mathcal{B}$ contains a complete bipartite subgraph $\mathcal{H} = (\mathcal{X}_H, \mathcal{Y}_H, \mathcal{E}_H)$ such that $|\mathcal{X}_H| = |\mathcal{Y}_H| = \eta$. Then, by our construction, for each node $u_j \in \mathcal{Y}_H$, we have a star-shaped subgraph in $\mathcal{G}$ which includes all the nodes in $\mathcal{X}_H$ and a node s. Let $\mathcal{C}$ be the star-shaped subgraph induced by $\mathcal{X}_H \cup \{s\}$. Clearly, $\mathcal{C}$ is a 1-core. For each $u_j \in \mathcal{Y}_H$, the maximal (θ, k)-persistent-core interval of $\mathcal{C}$ is $[2j - 2, 2j]$. Since $|\mathcal{Y}_H| = \eta$, we have η different and non-overlapped maximal (θ, k)-persistent-core intervals with the same length 2. Therefore, the core persistence of $\mathcal{C}$ is $\eta+1$. As a consequence, we obtain an Yes-instance of our problem, because $\mathcal{C}$ has $\eta+1$ nodes and its core persistence is no smaller than $\tau = \eta+1$.

Second, suppose that $\mathcal{G}$ consists of a temporal subgraph $\mathcal{C}$ with $\eta + 1$ nodes and its core persistence (core parameter $k = 1$) is no less than $\eta + 1$. Recall that $\mathcal{G}$ is constructed by a set of star-shaped subgraphs at different timestamps. For a star-shaped subgraph $\mathcal{S}$, we call an interval $[t_s, t_e]$ the maximal θ-persistent interval of $\mathcal{S}$ if and only if the following two conditions hold: (1) $\mathcal{S}$ appears in every θ-length subintervals and (2) there does not exist a super-interval that satisfies (1). Clearly, for each star-shaped subgraph $\mathcal{S}_i$, the maximal θ-persistent interval is $[2i - 2, 2i]$ $(\theta = 1)$. Since all the maximal 1-persistent intervals of the star-shaped subgraphs are non-overlapped and the core persistence of $\mathcal{C}$ is no less than $\eta + 1$, $\mathcal{C}$ must be contained in at least η different star-shaped subgraphs. On the other hand, since the core persistence of $\mathcal{C}$ is no less than $\eta + 1$, the projected graph of $\mathcal{C}$ is connected. By our construction, we can conclude that $\mathcal{C}$ must contain the node s. By picking η star-shaped subgraphs that contains $\mathcal{C}$, we can recover a balanced complete bipartite subgraph that consists of η nodes on each side. Thus, we obtain an Yes-instance of the BBS problem. □

LEMMA 2. *Let $[t^u_{s_1}, t^u_{e_1}]$ and $[t^u_{s_2}, t^u_{e_2}]$ be two overlapped maximal (θ, k)-persistent-degree intervals of a node u, and $t^u_{e_1} < t^u_{e_2}$. Then, we have $t^u_{e_1} - t^u_{s_2} \leq \theta$.*

PROOF. The proof is similar to the proof of Lemma 1, thus we omit for brevity. □

LEMMA 3. *Let $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ be a temporal subgraph of $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$. Then, for any node $u \in \mathcal{V}_1$, we have $f(u, \theta, k, \mathcal{G}_1) \leq f(u, \theta, k, \mathcal{G}_2)$.*

PROOF. The proof can be easily obtained by definition, thus we omit the details for brevity. □

LEMMA 4. *Suppose that there is a (θ, τ)-persistent k-core $\mathcal{C}$ in the temporal graph $\mathcal{G}$. Then, for every node $u \in \mathcal{C}$, we has $f(u, \theta, k, \mathcal{C}) \geq \tau$.*

PROOF. The proof can be easily obtained by the definition of (θ, τ)-persistent k-core, thus we omit it for brevity. □

THEOREM 2. *For each $\mathcal{MI}_u(i) = (t_s, t_e)$ and $\mathcal{D}_u(i)$ computed by Algorithm 1, the interval $(t_s - \theta, t_e)$ is a meta-interval. Moreover, in any θ-length subinterval of $(t_s - \theta, t_e)$, u has the same degree $\mathcal{D}_u(i)$.*

PROOF. Since t_s and t_e are two adjacent timestamps in the array $\mathcal{T}$, $t_e - t_s \leq \theta$ and there does not exist a timestamp $t \in \mathcal{T}$ such that $t_s < t < t_e$. As a consequence, there is also no timestamp $t' \in \mathcal{T}$ such that $t_s - \theta < t' < t_e - \theta$. This is because if there is a timestamp t_i with $t_s - \theta < t_i < t_e - \theta$, then $t_i + \theta$ must fall into the range (t_s, t_e), which is a contradiction. Therefore, we can conclude that there is no temporal edge with timestamp t falling in the range (t_s, t_e) or $(t_s - \theta, t_e - \theta)$. By Algorithm 1, $\mathcal{D}_u(i)$ is the prefix sum computed up to $\mathcal{T}(i)$, thus there are $\mathcal{D}_u(i)$ *surviving* edges in the interval $[t_e - \theta, t_s]$. Since no edge exists in (t_s, t_e) and $(t_s - \theta, t_e - \theta)$, we can conclude that u has a degree $\mathcal{D}_u(i)$ in any θ-length subinterval of $(t_s - \theta, t_e)$. On the other hand, the length of the interval $(t_s - \theta, t_e)$ is clearly larger than θ. Therefore, $(t_s - \theta, t_e)$ is a meta-interval by Definition 6. □

THEOREM 3. *Both the time and space complexity of Algorithm 1 to compute the meta-interval decomposition for all nodes $u \in \mathcal{G}$ are $O(m)$, where m is the number of temporal edges.*

PROOF. For each node u, Algorithm 1 only need to traverse the temporal neighborhood $\mathcal{N}_u(\mathcal{G})$ to compute the meta-intervals of u. Clearly, in lines 2-4, the time complexity is $O(|\mathcal{N}_u(\mathcal{G})|)$. In line 5, the algorithm can sort $\{\mathcal{T}_1, \mathcal{T}_2\}$ in linear time as the timestamps are integers. Similarly, for other steps (lines 6-15), the time cost is also $O(|\mathcal{N}_u(\mathcal{G})|)$. As a result, for all nodes, Algorithm 1 takes $O(m)$ time in total. For the space complexity, the algorithm only need to maintain several auxiliary arrays $\mathcal{D}_u$ and $\mathcal{MI}_u$ which use linear space cost in total. □

LEMMA 5. *For a parameter k and $i \leq h-1$, if both $\mathcal{D}_u(i) \geq k$ and $\mathcal{D}_u(i+1) \geq k$ hold, u has a degree no less than k in any θ-length subinterval of $[t_{s_i} - \theta, t_{e_{i+1}}]$.*

PROOF. Since $\mathcal{D}_u(i) \geq k$ ($\mathcal{D}_u(i+1) \geq k$), the degree of u in any θ-length subinterval of the interval $(t_{s_i} - \theta, t_{e_i})$ (interval $(t_{s_{i+1}} - \theta, t_{e_{i+1}})$) is no less than k. Recall that by our construction we have $t_{e_i} = t_{s_{i+1}}$ (see line 14 in Algorithm 1). The length of the overlapped interval of $(t_{s_i} - \theta, t_{e_i})$ and $(t_{e_i} - \theta, t_{e_{i+1}})$ is equal to θ. Thus, we are capable of merging these two intervals. Moreover, it is easy to see that in the merged interval $[t_{s_i} - \theta, t_{e_{i+1}}]$, u has a degree no less than k in any θ-length subinterval. □

THEOREM 4. *If $\mathcal{D}_u(i) \geq k$ for all $i = l, \cdots, r$ ($l \leq r$), $\mathcal{D}_u(l-1) < k$ and $\mathcal{D}_u(r+1) < k$ hold, then $[t_{s_l} - \theta, t_{e_r}]$ is a maximal (θ, k)-persistent-degree interval of u.*

PROOF. First, by Lemma 5, we can derive that u has a degree no less than k in any θ-length subinterval of the interval $[t_{s_l} - \theta, t_{e_r}]$. Since $\mathcal{D}_u(l-1) < k$ and $\mathcal{D}_u(r+1) < k$, the θ-persistent degree of u in $(t_{s_{l-1}} - \theta, t_{e_{l-1}})$ (if $(t_{s_{l-1}}, t_{e_{l-1}}) \in \mathcal{MI}_u$) and $(t_{s_{r+1}} - \theta, t_{e_{r+1}})$ (if $(t_{s_{r+1}}, t_{e_{r+1}}) \in \mathcal{MI}_u$) is smaller than k. By our construction, we have $t_{e_{l-1}} = t_{s_l}$ and $t_{s_{r+1}} = t_{e_r}$. There is no temporal edge falling into the open intervals $(t_{s_{l-1}} - \theta, t_{s_l} - \theta)$ and $(t_{e_r}, t_{e_{r+1}})$. Since u's θ-persistent degree is smaller than k in the intervals $(t_{s_{l-1}} - \theta, t_{s_l})$ and $(t_{e_r} - \theta, t_{e_{r+1}})$, we cannot extend the interval $[t_{s_l}, t_{e_r}]$. As a result, $[t_{s_l}, t_{e_r}]$ is the maximal (θ, k)-persistent-degree interval of u. □

THEOREM 5. *For a node u, if $f(u, \theta, k, \mathcal{G}) > \theta$, we have $f(u, \theta, k, \mathcal{G}) = \sum_{\mathcal{D}_u(i) \geqslant k} (t_{e_i} - t_{s_i}) + \theta$, where $(t_{s_i}, t_{e_i}) \in \mathcal{MI}_u$.*

PROOF. The proof can be easily obtained by Theorem 4 and Definition 5, thus we omit the details for brevity. □

THEOREM 6. *For an interval $\tilde{\mathcal{MI}}_u(i) = [t_i, t_i]$, if $\tilde{\mathcal{D}}_u(i) = k$, $\mathcal{D}_u(i-1) < k$ and $\mathcal{D}_u(i) < k$, then $[t_i - \theta, t_i]$ is a θ-length maximal (θ, k)-persistent-degree interval of u.*

PROOF. First, if $\tilde{\mathcal{D}}_u(i) = k$, u has a degree k in $[t_i - \theta, t_i]$. Since $\mathcal{D}_u(i-1) < k$ and $\mathcal{D}_u(i) < k$, the θ-persistent degree of u in the interval $(t_{s_{i-1}} - \theta, t_{e_{i-1}})$ (if $(t_{s_{i-1}}, t_{e_{i-1}})$ exists in $\mathcal{MI}_u$) and $(t_{s_i} - \theta, t_{e_i})$ is smaller than k. Note that by our construction (see line 14 in Algorithm 1), we have $t_{s_i} = t_i$ and $t_{e_{i-1}} = t_i$ (if it exists). Moreover, there does not exist a temporal edge falling into the open intervals $(t_{s_{i-1}} - \theta, t_{e_{i-1}} - \theta)$ and (t_{s_i}, t_{e_i}). Since u's θ-persistent degree is smaller than k in the intervals $(t_i - \theta, t_{e_i})$ and $(t_{s_{i-1}} - \theta, t_i)$, we cannot extend the interval $[t_i - \theta, t_i]$. Thus, $[t_i - \theta, t_i]$ is the maximal (θ, k)-persistent-degree interval of u. □

LEMMA 6. *After deleting a temporal edge $e = (u, v_i, t_i)$ from $\mathcal{N}_u(\mathcal{G})$, we decrease $\mathcal{D}_u(j)$ by 1 for all $j = l, \cdots, r$, and keep $\mathcal{D}_u(j)$, for all $j \notin [l, r]$, unchanged. Based on the updated $\mathcal{D}_u$ array, we have $f(u, \theta, k, \mathcal{G} \backslash \{e\}) = \sum_{\mathcal{D}_u(i) \geqslant k} (t_{e_i} - t_{s_i}) + \theta$.*

PROOF. Since the deletion of e decreases the θ-persistent degree of u by 1 in each meta-interval in $\mathcal{I} = \{(t_{s_l} - \theta, t_{e_l}), \cdots, (t_{s_r} - \theta, t_{e_r})\}$, we decrease $\mathcal{D}_u(j)$ by 1 for all $j = l, \cdots, r$. In other meta-intervals, we keep the θ-persistent degree of u unchanged. The updated degree persistence $f(u, \theta, k, \mathcal{G} \backslash \{e\})$ can be computed by Theorem 5 using the updated $\mathcal{D}_u$ array. □

LEMMA 7. *After inserting a temporal edge $e = (u, v_i, t_i)$ back into $\mathcal{N}_u(\mathcal{G}) \backslash \{e\}$, we increase $\mathcal{D}_u(j)$ by 1 for all $j = l, \cdots, r$ and keep $\mathcal{D}_u(j)$, for all $j \notin [l, r]$, unchanged. Based on the updated $\mathcal{D}_u$ array, we have $f(u, \theta, k, \mathcal{G}) = \sum_{\mathcal{D}_u(i) \geqslant k} (t_{e_i} - t_{s_i}) + \theta$.*

PROOF. The proof is similar to the proof Lemma 6, we omit the details for brevity. □

THEOREM 7. *The time and space complexity of Algorithm 2 is $O(\theta m)$ and $O(m)$ respectively.*

PROOF. First, in lines 2-6, the algorithm takes $O(m)$ time to compute the meta-interval composition for all nodes. Second, since each node is pushed into $\mathcal{Q}$ at most once, the number of elements in $\mathcal{Q}$ is bounded by $O(n)$. For each node $u \in \mathcal{Q}$, the algorithm visits its temporal neighborhood once in line 9. For each temporal edge, the algorithm takes $O(\theta)$ time to update the degree persistence. Therefore, in lines 7-19, the total time complexity is $O(\theta m)$. For the space complexity, we can easily show that it is $O(m)$, as the algorithm only need to maintain several linear-size arrays. □

THEOREM 8. *Let $N_R = \sum_{u \in \{\mathcal{R}\}} N_u$. The time complexity of Algorithm 3 is $O(|\mathcal{R}| N_R)$.*

PROOF. In each iteration, the time complexity is bounded by $O(N_R)$, as there are at most $O(N_R)$ common meta-intervals generated by the algorithm. As a consequence, the total time cost is $O(|\mathcal{R}| N_R)$. □

THEOREM 9. *The time complexity of Algorithm 4 is $O(\theta |\mathcal{N}_u(\mathcal{G})|)$.*

PROOF. Since the lifespan of each temporal edge is bounded by $O(\theta)$, the cost of the "while" loop is $O(\theta)$. There are $O(|\mathcal{N}_u(\mathcal{G})|$ temporal edges, thus the total time cost is $O(\theta |\mathcal{N}_u(\mathcal{G})|)$. □

A.2 The persistent clique model

As discussed in Section 2, the temporal clique model [26, 12] cannot fully capture the persistence of a community. Similar to our (θ, τ)-persistent k-core model, we can revise the temporal clique model [26, 12] to handle this limitation. Specifically, similar to Definition 1, we first define the maximal θ-persistent-clique interval by replacing the condition (1) in Definition 1 with a clique constraint. The formal definition is given as follows.

DEFINITION 1. (Maximal θ-persistent-clique interval) *Let $\mathcal{C}$ be an induced temporal subgraph of $\mathcal{G}$. An interval $[t_s, t_e]$ with $t_e - t_s \geq \theta$ is called a maximal θ-persistent-clique interval for $\mathcal{C}$ if and only if the following two conditions hold. (1) For any $t \in [t_s, t_e - \theta]$, the projected graph of $\mathcal{C}$ over the interval $[t, t + \theta]$ is a clique. (2) There is no super-interval of $[t_s, t_e]$ such that (1) holds.*

Clearly, there may be several maximal θ-persistent-clique intervals for an induced temporal subgraph $\mathcal{C}$. Similar to Definition 2, we define the clique persistence $H(\theta, C)$ using the same method as used in Eq. (1) to aggregate all maximal θ-persistent-clique intervals. Based on the clique persistence, we introduce the definition of the (θ, τ)-persistent clique as follows.

DEFINITION 2. ((θ, τ)-persistent-clique) *For an induced temporal subgraph $\mathcal{C}$, parameters θ and τ, $\mathcal{C}$ is a (θ, τ)-persistent clique if and only if it meets the following properties.*

- Persistent clique property*: $H(\theta, \mathcal{C}) \geq \tau$;*
- Maximal property*: there does not exist an induced temporal subgraph $\mathcal{C}'$ that contains $\mathcal{C}$ and also satisfies the persistent clique property.*

In real-world applications, we are typically interested in finding large persistent cliques. The following lemma shows that all the (θ, τ)-persistent cliques with size no less than k must be contained in the reduced temporal graph computed by the TGR algorithm (Algorithm 2) with parameters $\theta, k-1$, and τ.

LEMMA 8. *Let $\tilde{\mathcal{C}}$ be the reduced temporal graph computed by Algorithm 2 with parameters θ, $k-1$, and τ. Then, every (θ, τ)-persistent clique with size no less than k must be contained in $\tilde{\mathcal{C}}$.*

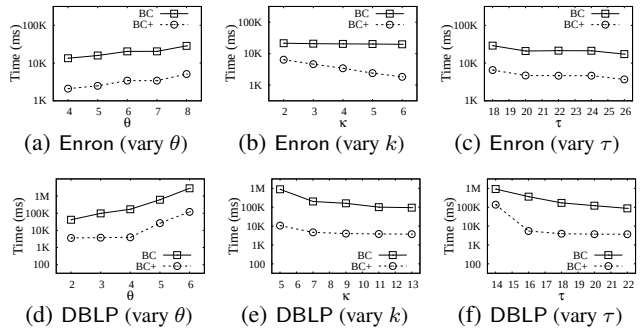
PROOF. By Definition 2, every node in a (θ, τ)-persistent clique with size no less than k must persistently maintain the property of "degree no less than $k-1$" in the maximal θ-persistent-clique intervals with total length no smaller than τ (i.e., the persistent-degree property). Clearly, the reduced temporal graph returned by Algorithm 2 with parameters $\theta, k-1$, and τ contains all the nodes that satisfy such a persistent-degree property. As a result, all the (θ, τ)-persistent cliques must be contained in $\tilde{\mathcal{C}}$. □

With Lemma 8, we can first prune the temporal graph $\mathcal{G}$ using Algorithm 2, and then slightly modify the state-of-the-art temporal clique enumeration algorithm [12] to enumerate all the (θ, τ)-persistent cliques with size no less than k. Below, we empirically show that the proposed TGR technique drastically reduces the search time of the persistent clique search algorithm.

Efficiency of the persistent clique search algorithms. We implement two different algorithms to find (θ, τ)-persistent cliques with size no less than $k-1$: BC and BC+. BC is a straightforward modification of the state-of-the-art temporal clique enumeration (TCE) algorithm [12] (with size and presentence constraints), and BC+ denotes the BC algorithm equipped with the TGR technique. Both BC and BC+ have three parameters θ, k, and τ, and the parameter settings are reported in Table 2. Fig. 9 depicts the efficiency of BC and BC+ with varying parameters in Enron and DBLP datasets. Similar results can also be observed in the other datasets. As can be seen, both the running time of BC and BC+ increase with growing θ, because the number of persistent cliques increases with increasing θ. Both the running time of BC and BC+ decrease with increasing k or τ. The reason is that for a large k or τ, the persistent clique constraint is strong, thus the number of persistent cliques decreases. We can also observe that BC is costly in DBLP, and BC+ is at least one order of magnitude faster than BC. For example, in Fig. 9(d), when $\theta = 4$, BC consumes 170 seconds, while BC+ takes only 4 seconds. These results confirm that our TGR technique indeed substantially speeds up the persistent clique search algorithm.

A.3 Additional case studies

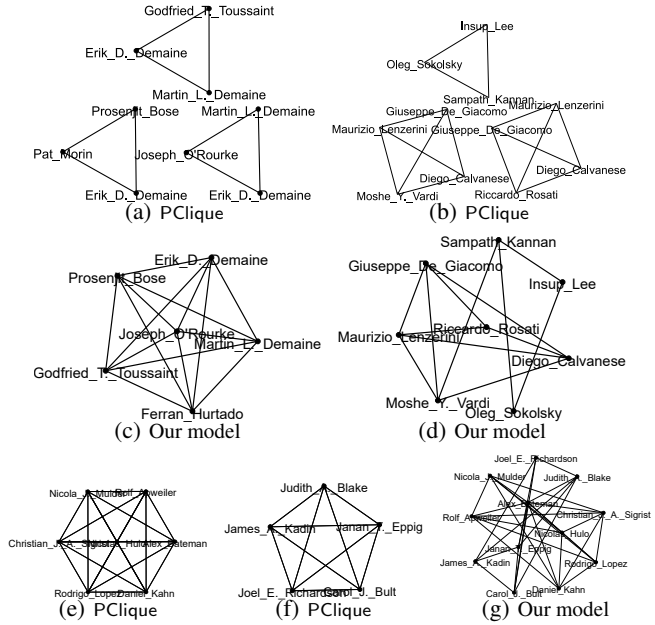
Additional case studies in DBLP. Since KCore is ineffective, we focus mainly on comparing our model and PClique. Figs. 10(a-d) depict the results obtained by our model and PClique using parameters $\theta = 2$, $k = 2$, and $\tau = 17$. Similar to the previous results, our model performs much better than the PClique to identify persistent communities. For example, in Fig. 10(a), PClique obtains three redundant triangle communities, whereas our model can obtain a large and compact community (Fig. 10(c)). In fact, all the researchers in these triangle communities persistently



(a) Enron (vary θ) (b) Enron (vary k) (c) Enron (vary τ)

(d) DBLP (vary θ) (e) DBLP (vary k) (f) DBLP (vary τ)

Figure 9: Efficiency of the persistent clique search algorithms



(a) PClique (b) PClique (c) Our model (d) Our model (e) PClique (f) PClique (g) Our model

Figure 10: Comparison of various models in DBLP. The results in Figs. (a-d) and Figs. (e-g) are obtained using parameters ($\theta = 2$, $k = 2$, $\tau = 17$) and ($\theta = 3$, $k = 4$, $\tau = 18$) respectively.

coauthor papers in the last few decades. Using our model, we can reveal such a persistent community. Similar results can also be obtained using parameters $\theta = 3$, $k = 4$, and $\tau = 18$ (see Figs. 10(e-g)).

Case studies in Hospital. Here we perform case studies in Hospital dataset. The Hospital dataset is a temporal network of face-to-face contacts between patients, patients and health-care workers (including nurses, doctors, and administrative staffs) in a hospital in France. The dataset contains 75 nodes (46 health-care workers and 29 patients), as well as 32,424 temporal edges. Each temporal edge denotes a close-range interaction (contact) between individuals in the hospital, which is detected by wearable sensors carried by the individuals [24]. The time scale of this dataset is 20 seconds, and the whole time interval of the temporal network is around 5 days [24].

We first make use of the parameters $\theta = 60$ (seconds), $k = 3$, and $\tau = 200$ for case study. The resulting community obtained by KCore includes 62 nodes, which clearly cannot reflect a persistent community. Using our model, we can obtain 17 communities. We are able to get 19 communities using PClique. Some of our results are depicted in Fig. 11. Similar to the results in DBLP, both our model and PClique can find persistent and compact communities. The result obtained by PClique (Figs. 11(a-b) and (d-e)) tend to

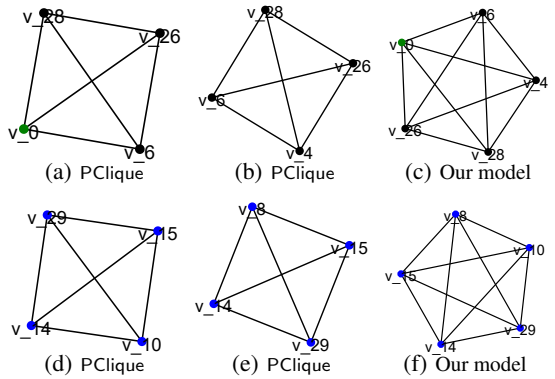


Figure 11: Comparison of various models in Hospital **($\theta = 60$, $k = 3$, $\tau = 200$). Black circle denotes nurse, blue circle denotes doctor, and green circle stands for administrative staff.**

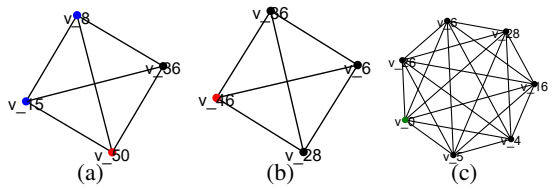


Figure 12: Different semantics of contact revealed by our model in Hospital**. Figs. (a-b) are obtained using ($\theta = 60, k = 3, \tau = 200$); Fig. (b) is also obtained using ($\theta = 3600, k = 3, \tau = 21000$); Fig. (c) is obtained using ($\theta = 900, k = 3, \tau = 3600$). Red circle represents patient, black circle denotes nurse, blue circle denotes doctor, and green circle stands for administrative staff.**

be a partial community computed by our model (Figs. 11(c) and (f)). This is because the *clique constraint* is stronger than our *core constraint*, and thus the persistent clique is contained in our community model.

Fig. 12 shows that our model can reveal different semantics of contact between patients and health-care workers. For example, in Fig. 12(a), the community contains two doctors, a patient, and a nurse. This community lasts round 4 minutes, which is probably to be an event that the doctors treat the patient with the help of a nurse. In Fig. 12(b), we find a community involving three nurses and a patient under two different parameter settings: ($\theta = 60, k = 3, \tau = 200$) and ($\theta = 3600, k = 3, \tau = 21000$). This community indicate that the patient could be in intensive care, as he/she persistently contacts with three nurses for a long time. In Fig. 12(c), the community involving one administrative staff and six nurses lasts more than one hour, which is likely to be a one-hour meeting among these individuals.

Case studies in HighSchool**.** Here we show case studies in HighSchool. HighSchool is a temporal network of contacts between students in a French high school. The dataset contains 181 students from five various classes and 45,047 temporal edges. Each temporal edge (u, v, t) describes a face-to-face contact between students u and v at time t. The time scale of this dataset is 20 seconds, and the whole time interval of the temporal network is around 8 days. More details of this dataset can be found in [9].

We first use the parameters $\theta = 3600$, $k = 4$, and $\tau = 7200$ for case study (as a typical course lasts two hours in this dataset [9, 25]). Under this parameter setting, KCore can obtain a community involving 157 students (nearly contains all students). Clearly, all those 157 students unlikely form a persistent community. This result further confirms that KCore is ineffective to identify persistent communities. Under this parameter setting, our model can obtain 14 different persistent communities, whereas PClique gets 11 communities. Similar to our previous results, PClique tends to find partial results of the persistent communities obtained by our model. For example, Figs. 13(a-b) depicts two communities computed by PClique which are a partial community obtained by our model as shown in Fig. 13(c). As depicted in Figs. 13(d-f), if we extend the parameter τ to 7500 seconds, PClique can only find one community, while our model can find two communities. The persistent community in Fig. 13(f) cannot be found by PClique.

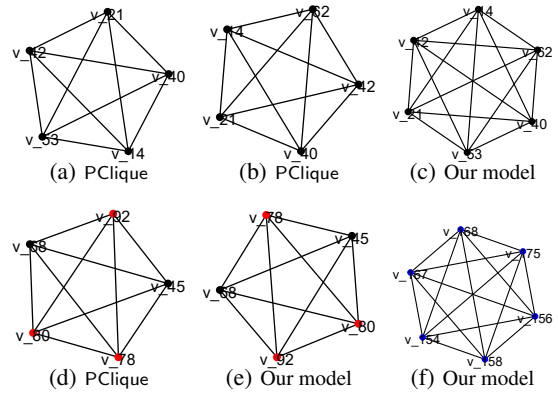


Figure 13: Comparison of various models in HighSchool**. Figs. (a-c) are obtained using ($\theta = 3600$, $k = 4$, $\tau = 7200$), and Fig. (d-f) are obtained using ($\theta = 3600$, $k = 4$, $\tau = 7500$) Students from different classes are labeled by various colors.**

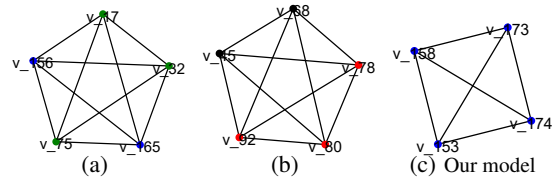


Figure 14: Different semantics of contact revealed by our model in HighSchool**. Figs. (a-b) are computed using parameters ($\theta = 30$, $k = 4$, $\tau = 120$), and Fig. (c) is obtained using parameters ($\theta = 3600$, $k = 3$, $\tau = 10800$)**

Our model can also reveal different semantics of contact between students. For example, in Fig. 14(a), the community contains 5 students of different classrooms during a time interval $[15:46, 15:48]$ (we can convert the timestamps to dates in this dataset.). This community is likely to be a group of friends that gathered during a break of a two-hour course. Similarly, in Fig. 14(b), the community involves five students of different classrooms during a time interval $[12:43, 12:45]$, which probably corresponds to a group gathering during the lunch break. The community in Fig. 14(c) contains four students of the same classrooms during more than 3 hours which is likely to be the signature of a three-hour lecture.